24D36I0710622049

EXP NO: 2(a) DATE: 23/6/26	8-BIT ADDER USING VERILOG HDL IN VIVADO

AIM:

To design and simulate an 8-bit binary adder using Verilog HDL in Vivado and verify the output.

SOFTWARE USED:

Vivado 2025.2

ALGORITHM:

Step 1: Open Vivado and create the Verilog file fulladder.v.

Step 2: Write the Verilog code for the Full Adder.

Step 3: Create the design source file adder8.v.

Step 4: Write the Verilog code for the 8-bit Adder.

Step 5: Create the simulation source file adder8_tb.v.

Step 6: Write the testbench code and apply different input values.

Step 7: Run the Behavioral Simulation in Vivado.

Step 8: Observe the simulation waveform and verify the Sum and Carry outputs.

PROGRAM:

DESIGN SOURCE:

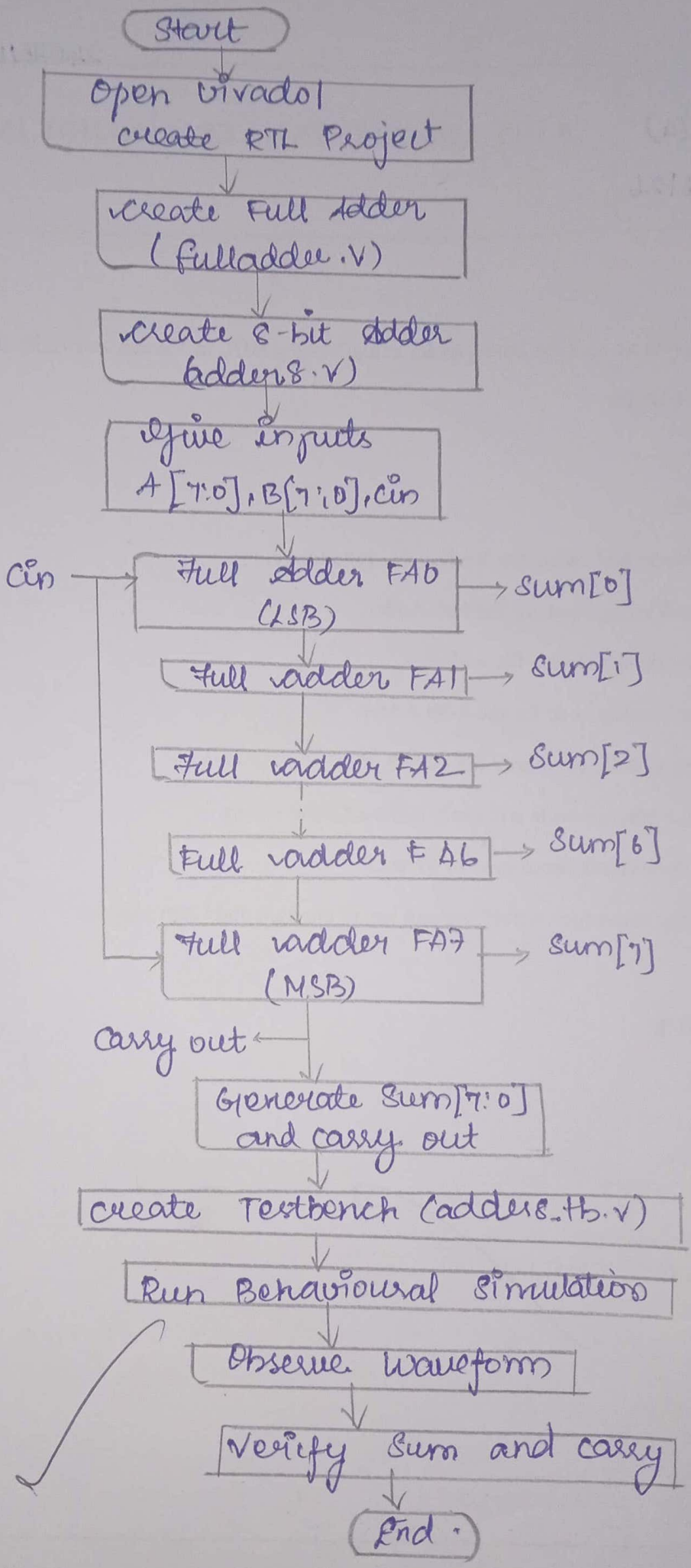
fulladder.v:

```
module fa(
    input a,b,c,
    output sum,carry
    );
    assign sum=a^b^c;
    assign carry=((a^b)&c)|(a&b);
endmodule
```

adder8.v:

```
module adder8(
    input [7:0] a,b,
    input cin,
```

FLOWCHART:


Start
Open Vivado create RTL Project
Create Full Adder (fulladder.v)
Create 8-bit adder (adder8.v)
Give inputs A[7:0], B[7:0], Cin
Cin
Full adder FA0 (LSB)
Sum[0]
Full adder FA1
Sum[1]
Full adder FA2
Sum[2]
Full adder FA6
Sum[6]
Full adder FA7 (MSB)
Sum[7]
Carry out
Generate Sum[7:0] and carry out
Create Testbench (adder8_tb.v)
Run Behavioural Simulation
Observe waveform
Verify Sum and carry
End

2403610710622041

```
    output [7:0] s,
    output carry
    );
    wire [6:0] c;

    fa z1(a[0],b[0],cin,s[0],c[0]);
    fa z2(a[1],b[1],c[0],s[1],c[1]);
    fa z3(a[2],b[2],c[1],s[2],c[2]);
    fa z4(a[3],b[3],c[2],s[3],c[3]);
    fa z5(a[4],b[4],c[3],s[4],c[4]);
    fa z6(a[5],b[5],c[4],s[5],c[5]);
    fa z7(a[6],b[6],c[5],s[6],c[6]);
    fa z8(a[7],b[7],c[6],s[7],carry);
endmodule
```

TESTBENCH CODE:

adder8_tb.v:

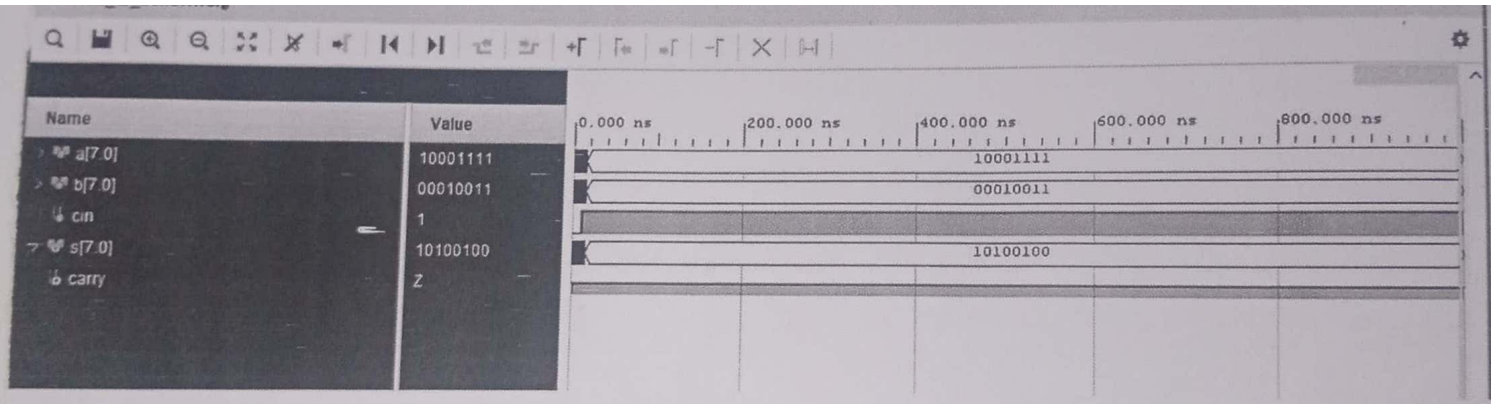
```
module adder8_tb;
reg [7:0]a;
reg [7:0]b;
reg cin;
wire [7:0]s;
wire carry;

adder8 dut(a,b,cin,s,carry);

initial
begin
    a=8'b00000000; b=8'b00001010; cin=1'b0;
    #10
    a=8'b00010001; b=8'b00101010; cin=1'b1;
    #5
    a=8'b00001111; b=8'b00110101; cin=1'b1;
end
endmodule
```

CONSTRAINTS:

```
set_property -dict { PACKAGE_PIN L5    IOSTANDARD LVCMOS33 } [get_ports { a[0] }];#LSB
set_property -dict { PACKAGE_PIN L4    IOSTANDARD LVCMOS33 } [get_ports { a[1] }];
set_property -dict { PACKAGE_PIN M4    IOSTANDARD LVCMOS33 } [get_ports { a[2] }];
set_property -dict { PACKAGE_PIN M2    IOSTANDARD LVCMOS33 } [get_ports { a[3] }];
set_property -dict { PACKAGE_PIN M1    IOSTANDARD LVCMOS33 } [get_ports { a[4] }];
set_property -dict { PACKAGE_PIN N3    IOSTANDARD LVCMOS33 } [get_ports { a[5] }];
set_property -dict { PACKAGE_PIN N2    IOSTANDARD LVCMOS33 } [get_ports { a[6] }];
set_property -dict { PACKAGE_PIN N1    IOSTANDARD LVCMOS33 } [get_ports { a[7] }];
set_property -dict { PACKAGE_PIN P1    IOSTANDARD LVCMOS33 } [get_ports { b[0] }];
set_property -dict { PACKAGE_PIN P4    IOSTANDARD LVCMOS33 } [get_ports { b[1] }];
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS33 } [get_ports { b[2] }];
```


Name
Value
a[7:0]
10001111
b[7:0]
00010011
cin
1
s[7:0]
10100100
carry
Z
0.000 ns
200.000 ns
400.000 ns
600.000 ns
800.000 ns
10001111
00010011
10100100

```
set_property -dict { PACKAGE_PIN R8    IOSTANDARD LVCMOS33 } [get_ports { b[3] }];
set_property -dict { PACKAGE_PIN N6    IOSTANDARD LVCMOS33 } [get_ports { b[4] }];
set_property -dict { PACKAGE_PIN T7    IOSTANDARD LVCMOS33 } [get_ports { b[5] }];
set_property -dict { PACKAGE_PIN P8    IOSTANDARD LVCMOS33 } [get_ports { b[6] }];
set_property -dict { PACKAGE_PIN M6    IOSTANDARD LVCMOS33 } [get_ports { b[7] }];#MSB

set_property -dict { PACKAGE_PIN J3    IOSTANDARD LVCMOS33 } [get_ports { sum[0]}];#LSB
set_property -dict { PACKAGE_PIN H3    IOSTANDARD LVCMOS33 } [get_ports { sum[1] }];
set_property -dict { PACKAGE_PIN J1    IOSTANDARD LVCMOS33 } [get_ports { sum[2] }];
set_property -dict { PACKAGE_PIN K1    IOSTANDARD LVCMOS33 } [get_ports { sum[3] }];
set_property -dict { PACKAGE_PIN L3    IOSTANDARD LVCMOS33 } [get_ports { sum[4] }];
set_property -dict { PACKAGE_PIN L2    IOSTANDARD LVCMOS33 } [get_ports { sum[5] }];
set_property -dict { PACKAGE_PIN K3    IOSTANDARD LVCMOS33 } [get_ports { sum[6] }];
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { sum[7] }];
set_property -dict { PACKAGE_PIN K5    IOSTANDARD LVCMOS33 } [get_ports { carry }];
```

RESULT:

Thus the 8-bit adder successfully added two 8-bit binary numbers. The Sum and Carry outputs matched the expected results.

(23)
2403610710622049

EXP NO: 2(b) DATE: 23/6/26	2X2 MULTIPLIER USING VERILOG HDL IN VIVADO

AIM:

To design and simulate a 2x2 binary multiplier using Verilog HDL in Vivado and verify the output.

SOFTWARE USED:

Vivado 2025.2

ALGORITHM:

Step 1: Open Vivado and create a new RTL project.

Step 2: Create the design source file mul.v.

Step 3: Write the Verilog code for the 2x2 multiplier.

Step 4: Create the simulation source file mul_tb.v.

Step 5: Write the testbench code and apply different input values.

Step 6: Run the Behavioral Simulation in Vivado.

Step 7: Observe the waveform and verify product output.

PROGRAM:

DESIGN SOURCE:

mul.v:

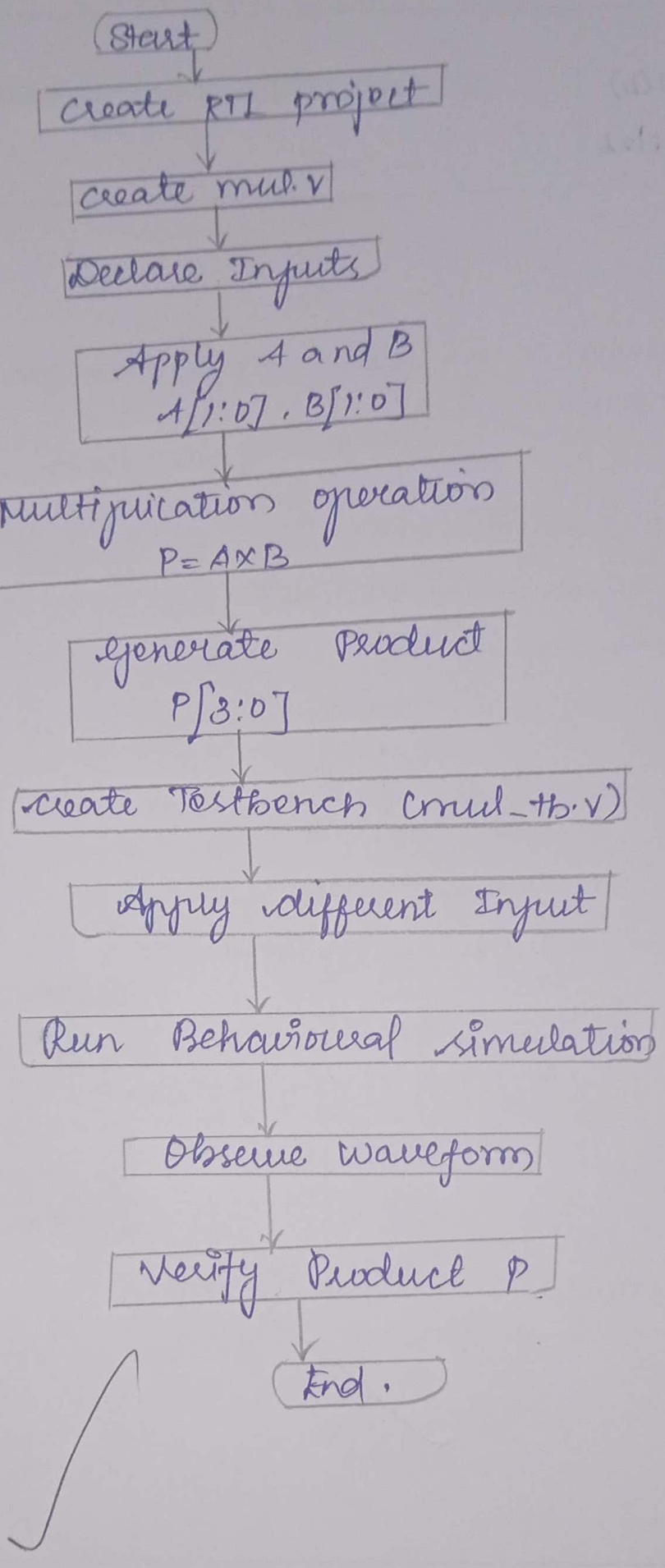
```
module mul(input[1:0]A,input[1:0]B,output [3:0]P);
assign P = A * B;
endmodule
```

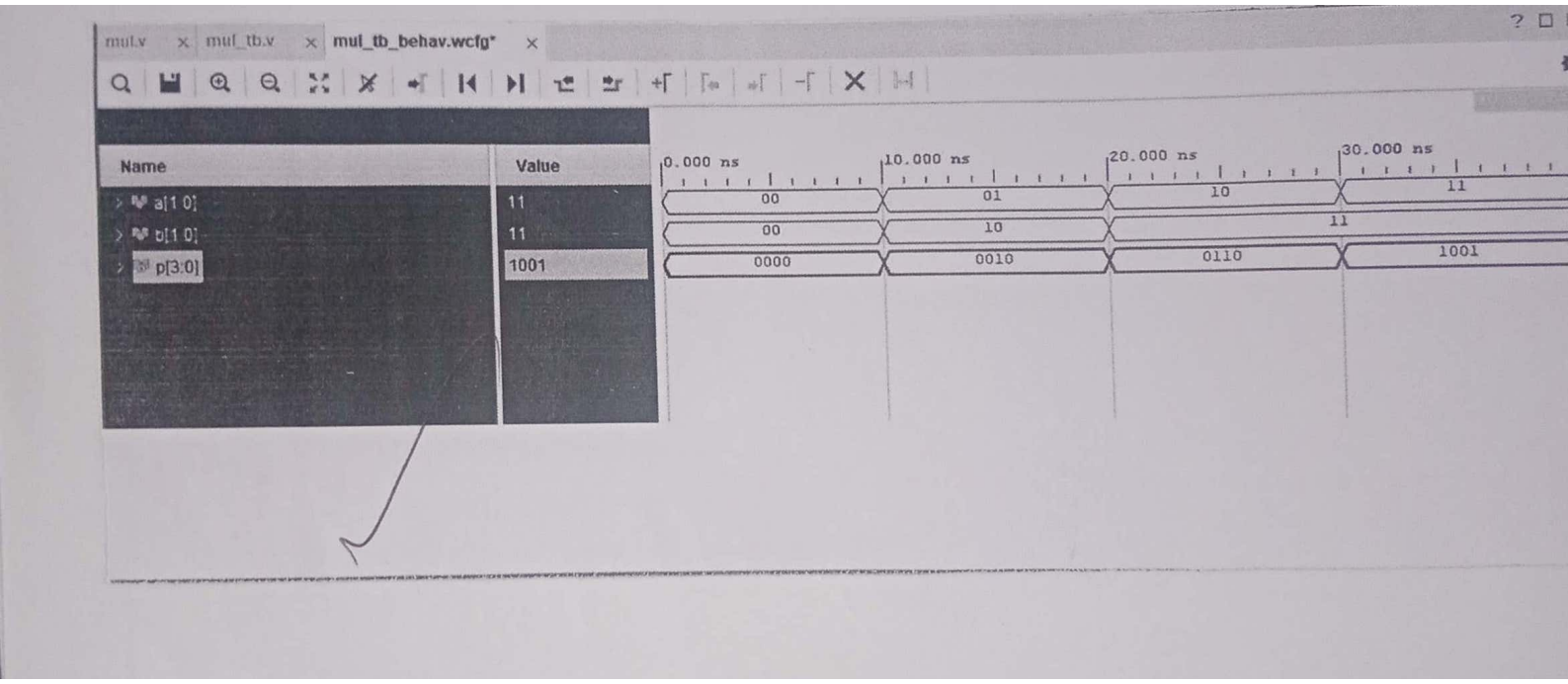
TESTBENCH CODE:

mul_tb.v:

```
module mul_tb;
reg[1:0]A;
reg[1:0]B;
wire[3:0]P;
mul dut(A(A),B(B),P(P));
```

FLOWCHART


Start
Create RTL project
create mul.v
Declare Inputs
Apply A and B
A[1:0], B[1:0]
Multiplication operation
P = A × B
Generate Product
P[3:0]
Create Testbench (mul_tb.v)
Apply different Input
Run Behavioural simulation
Observe waveform
Verify Product P
End.


mul.v
mul_tb.v
mul_tb_behav.wcfg*
Name
Value
a[1 0]
b[1 0]
p[3:0]
11
11
1001
0.000 ns
10.000 ns
20.000 ns
30.000 ns
00
01
10
11
00
10
11
0000
0010
0110
1001

```
initial
begin
    A = 2'b00; B = 2'b00;
    #10;
    A = 2'b01; B = 2'b01;
    #10;
    A = 2'b10; B = 2'b10;
    #10;
    A = 2'b11; B = 2'b11;
    #10;
    A = 2'b10; B = 2'b01;
    #10;
    A = 2'b11; B = 2'b10;
    #10;
    $finish;
end
endmodule
```

RESULT:

Thus the 2x2 binary multiplier was successfully designed, simulated and verified using Verilog HDL in Vivado.

(87)
2403610710622049

EXP NO: 2(c) DATE: 23/6/26	8:1 MULTIPLEXER USING 2:1MULTIPLEXER USING VERILOG HDL IN VIVADO

AIM:

To design and simulate on 8:1 multiplexer using 2:1 multiplexers using Verilog HDL in Vivado and verify the output.

SOFTWARE USED:

Vivado 2025.2

ALGORITHM:

Step 1: Open Vivado and create a new RTL project.

Step 2: Create the design source file mux2x1.v.

Step 3: Create the design source file mux8x1.v.

Step 4: Create the simulation source file mux8x1_tb.v.

Step 5: Write the testbench code and apply different input values.

Step 6: Run the Behavioral Simulation in Vivado.

Step 7: Observe the waveform and verify product output.

PROGRAM:

DESIGN SOURCE:

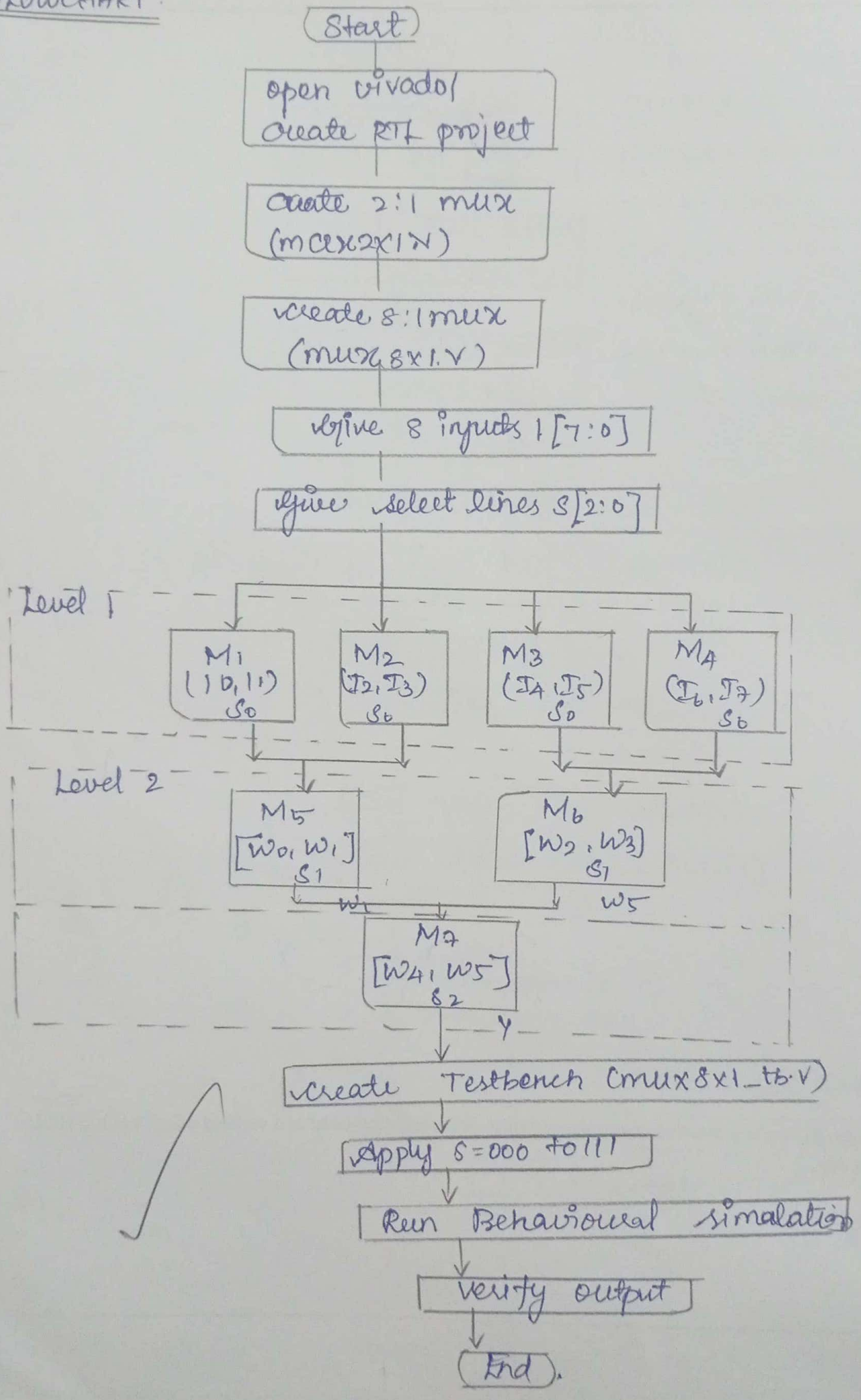
mux2x1.v:

```
module mux2x1(input I0, input I1, input S, output Y);
assign Y = S?I1:I0;
endmodule
```

mux8x1.v:

```
module mux8x1(input[7:0]I,input[2:0]S,output Y);
wire W0, W1, W2, W3;
wire W4, W5;
mux2x1 M1(I[0], I[1], S[0], W0);
mux2x1 M2(I[2], I[3], S[0], W1);
mux2x1 M3(I[4], I[5], S[0], W2);
mux2x1 M4(I[6], I[7], S[0], W3);
```

FLOWCHART:


Start
open vivado/ create RTL project
create 2:1 mux (mux2x1.v)
create 8:1 mux (mux8x1.v)
give 8 inputs I[7:0]
give select lines S[2:0]
Level 1
M1 (I0, I1) S0
M2 (I2, I3) S0
M3 (I4, I5) S0
M4 (I6, I7) S0
Level 2
M5 [W0, W1] S1
M6 [W2, W3] S1
W4
W5
M7 [W4, W5] S2
Y
create Testbench (mux8x1_tb.v)
Apply S=000 to 111
Run Behavioural simulation
verify output
End

```
mux2x1 M5(W0, W1, S[1], W4);
mux2x1 M6(W2, W3, S[1], W5);
mux2x1 M7(W4, W5, S[2], Y);
endmodule
```

TESTBENCH CODE:

mux8x1_tb.v:

```
module mux8x1_tb;
reg [7:0] I;
reg [2:0] S;
wire Y;
mux8x1 dut(I(I),S(S),Y(Y));
initial begin
I = 8'b10101010;
S = 3'b000; #10;
S = 3'b001; #10;
S = 3'b010; #10;
S = 3'b011; #10;
S = 3'b100; #10;
S = 3'b101; #10;
S = 3'b110; #10;
S = 3'b111; #10;
$finish;
end
endmodule
```

RESULT:

Thus the 8:1 multiplexer using 2:1multiplexer was successfully designed, simulated and verified using Verilog HDL in Vivado.

(34)

2403610710622049

EXP NO: 3 DATE: 30/6/26	FIR FILTER USING VERILOG HDL IN VIVADO

AIM:

To design and simulate a FIR[Finite Impulse Response] using Verilog HDL in Vivado and verify the output.

SOFTWARE USED:

Vivado 2025.2

ALGORITHM:

Step 1: Open Vivado and create a new RTL project.

Step 2: Create the D flip flop module dff.v.

Step 3: Create the design source file fir.v.

Step 4: Create the simulation source file fir_tb.v.

Step 5: Write the testbench code and apply different input values.

Step 6: Run the Behavioral Simulation in Vivado.

Step 7: Observe the waveform and verify product output.

PROGRAM:

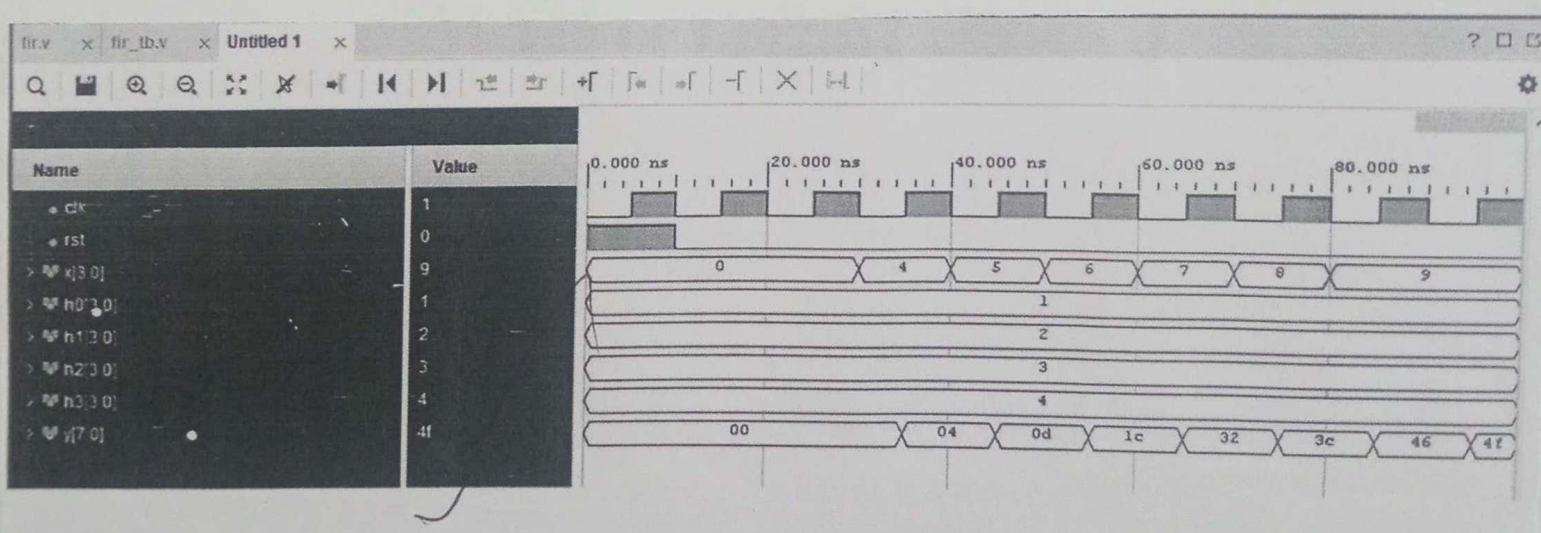
DESIGN SOURCE:

dff.v:

```
module diff(input clk,input rst,input[3:0]d,output reg[3:0]q);
always @(posedge clk)
begin
  if(rst)
    q<=4'b0000;
  else
    q <= d;
end
endmodule
```

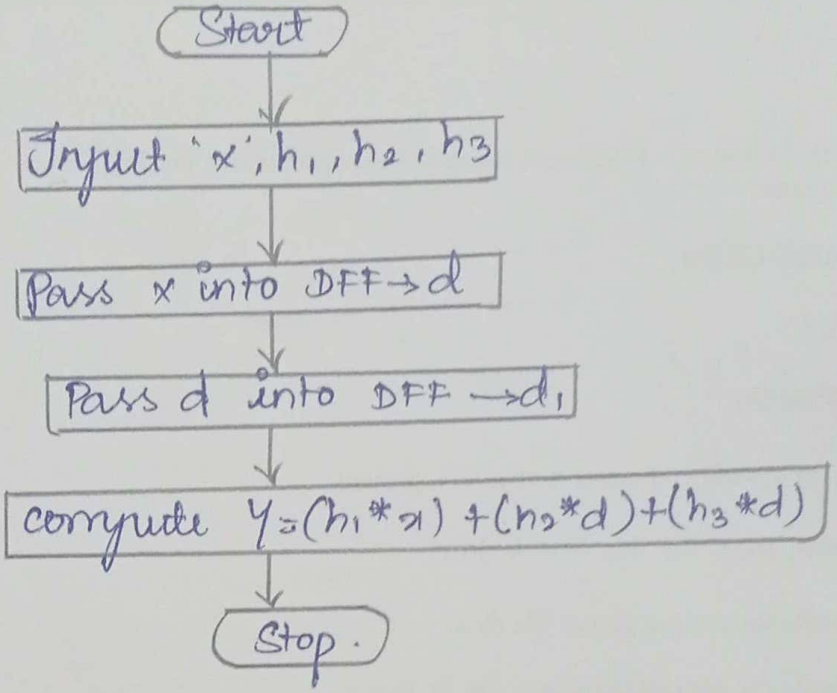
fir.v:

```
module fir(input clk,input rst,input[3:0]x,input[3:0]h0,input[3:0]h1,
           input[3:0]h2,output reg[7:0]y);
```

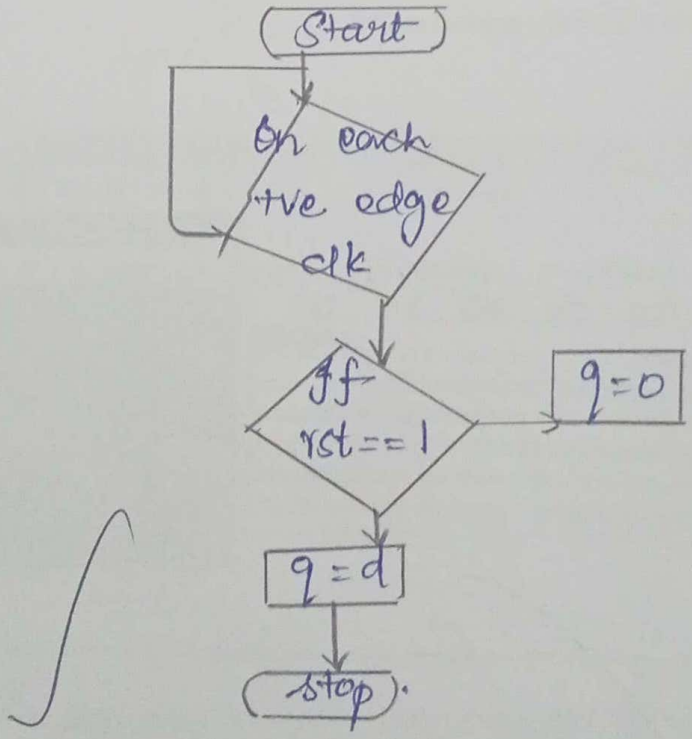

fir.v
fir_tb.v
Untitled 1
Name
Value
clk
rst
x[3:0]
h0[3:0]
h1[3:0]
h2[3:0]
h3[3:0]
y[7:0]
1
0
9
1
2
3
4
4f
0.000 ns
20.000 ns
40.000 ns
60.000 ns
80.000 ns
0
4
5
6
7
8
9
1
2
3
4
00
04
0d
1c
32
3c
46
4f

FLOWCHART

FIR :-



DFF :



```
wire [3:0] x1, x2;
wire [7:0] y1, y2, y3;
dff d1(clk, rst, x, x1);
dff d2(clk, rst, x1, x2);
assign y1 = x * h0;
assign y2 = x1 * h1;
assign y3 = x2 * h2;
always @(posedge clk)
begin
    if(rst)
        y <= 8'b00000000;
    else
        y <= y1 + y2 + y3;
end
endmodule
```

TESTBENCH CODE:

fir_tb.v:

```
module fir_tb;
reg clk;
reg rst;
reg [3:0] x;
reg [3:0] h0, h1, h2;
wire [7:0] y;
fir dut (clk(clk),rst(rst),x(x),h0(h0),h1(h1),h2(h2),y(y));
always #5 clk = ~clk;
initial
begin
  clk = 0;
  rst = 1;
  x = 0;
  h0 = 1;
  h1 = 2;
  h2 = 3;
  #10 rst = 0;
  #10 x = 4;
  #10 x = 5;
  #10 x = 6;
  #10 x = 7;
  #20 $finish;
end
endmodule
```

RESULT:

Thus the 2x2 binary multiplier was successfully designed, simulated and verified using Verilog HDL in Vivado.

EXP NO: 4

DATE: 17/8/26

DIGITAL CLOCK USING VERILOG HDL

AIM:

To design a digital clock using verilog HDL and implement it on FPGA.

APPARATUS REQUIRED:

Hardware: PC, FPGA kit

Software: Vivado 2025.2

ALGORITHM:

Step 1: Start the program

Step 2: Initialize the system

2.1: Reset the counters for second, minutes and hours to zero.

2.2: Initialize clock divider counter to generate slower clock.

Step 3: generate allow clock

3.1: Divide the main inputs clock to obtain two slower clocks.

3.1.1: One for time counting (second incument)

3.1.2: One for Multiplexing the I segment display digit.

Step 4: Time counting logic

4.1: One each vising edge of the clover clock for liming courtting.

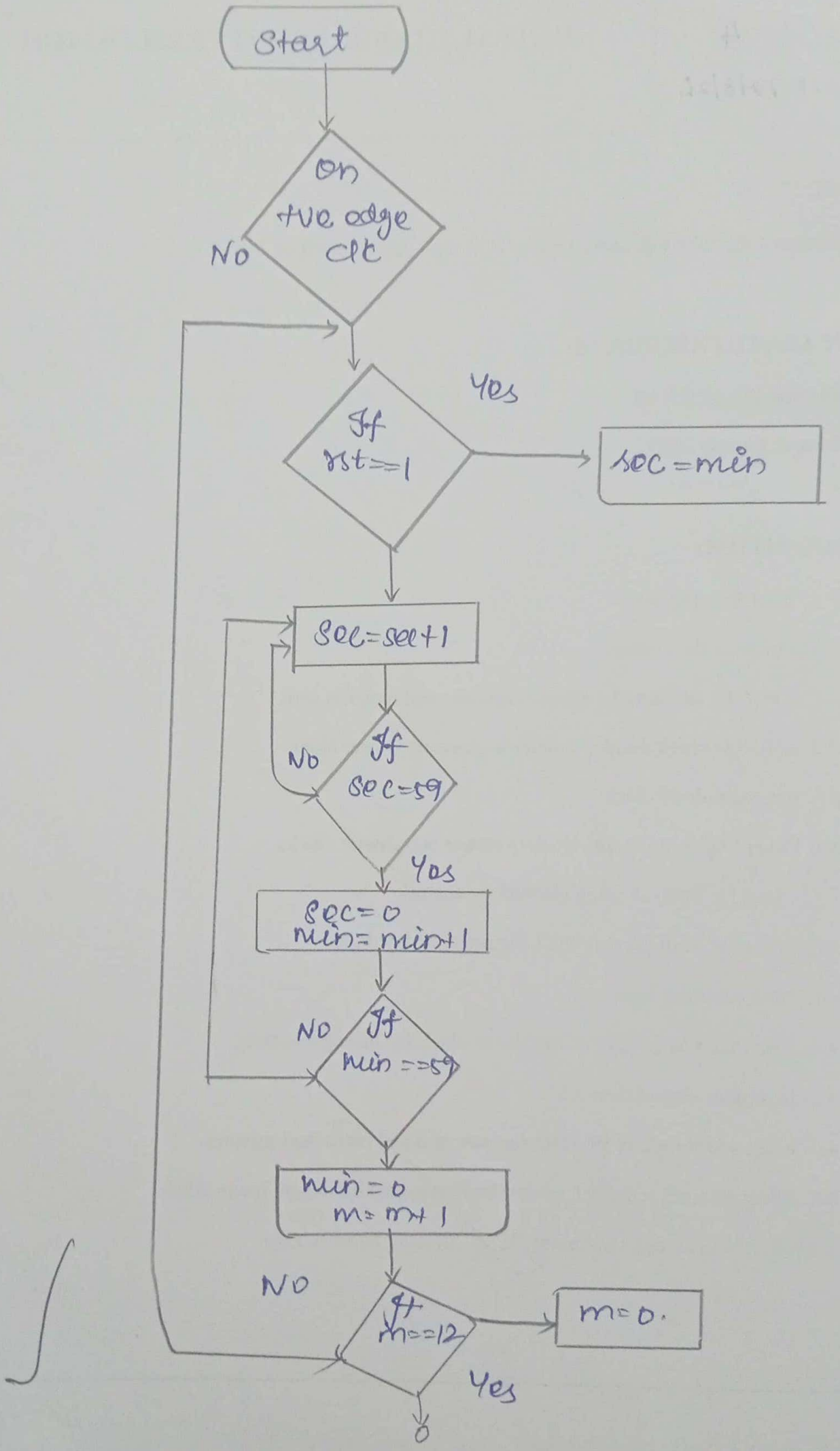
4.2: Incument second counter

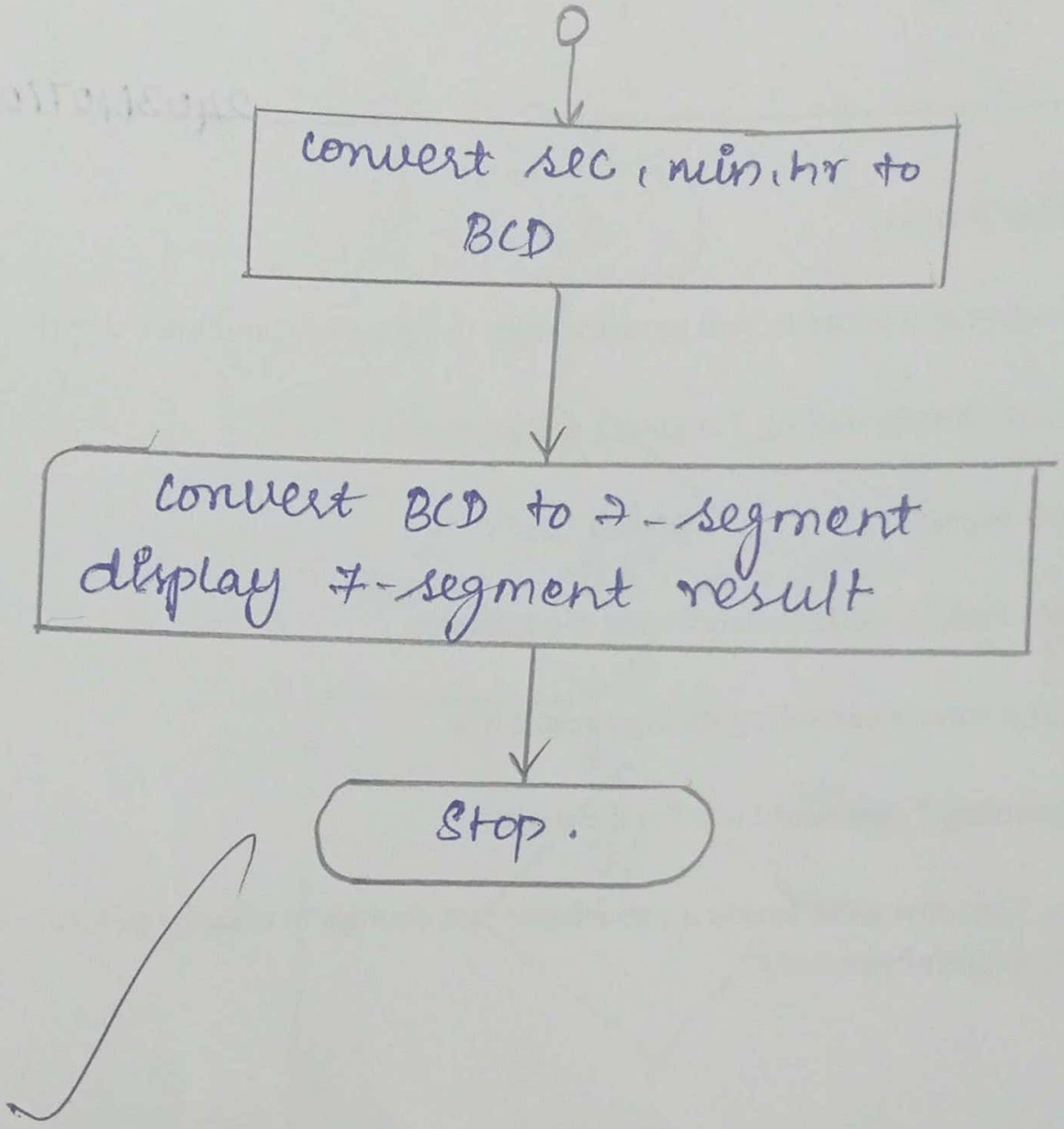
4.3: when second reach 59, reset seconds to o and increment minutes.

4.4: When minutes reach 59, reset minutes to oand Increment hours.AIM

4.5: when hours reach maximum (12 or 24) reset hours to zero

FLOWCHART :


Start
On +ve edge clk
No
If rst==1
Yes
sec = min
sec=sec+1
If sec=59
No
Yes
sec=0
min=min+1
If min==59
No
min=0
m=m+1
If m==12
NO
m=0.
Yes


convert sec, min, hr to BCD
convert BCD to 7-segment
display 7-segment result
Stop.

Step 5: Convert time to display format

5.1: Convert second, minutes and hours to two separate digit to each (tens and ones digit)

5.2: convert each digit is its corresponding 7 segment display code.

Step 6: Multiplex 7-segment display.

6.1 Use the second slow clock tit cycle through each digit display.

6.2: Activate one digit at a time by enabling its digit result line

6.3: Output the corresponding 7-segment code for the digit

6.4. Cycle continuously fast enough to create a persistance fast enough to create a persistance vision effect showing all digit simultaneously.

Step 7: Reset Handling

7.1: when reset in activated, clear all the counters to zero

Step 8: Stop the Program.

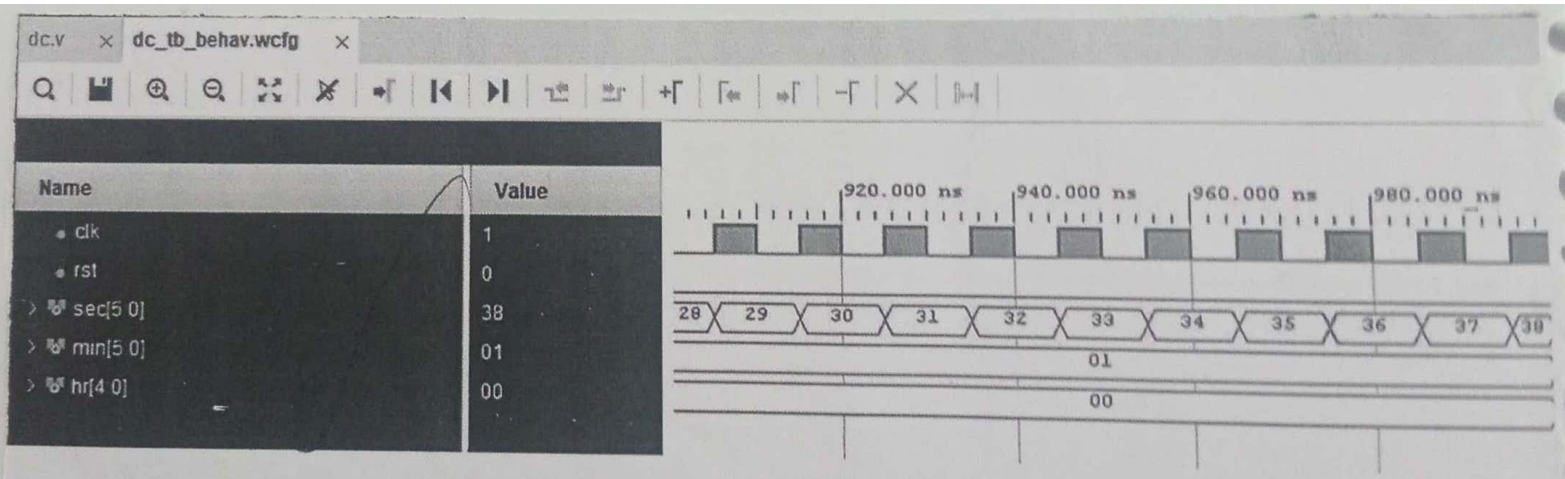
2403610710622049

PROGRAM:

```
//RTC_TOP
module rtc_top(clk,rst,sec,min,hr,sec_o,sec_t,
        min_o,min_t,hr_o,hr_t,sec_o_seg,sec_t_seg,
        min_o_seg,min_t_seg,hr_o_seg,hr_t_seg,data,digit);
input clk,rst;
output [5:0]min,sec;
output [3:0]hr,sec_o,sec_t,min_o,min_t,hr_o,hr_t,digit;
output [7:0] sec_o_seg,sec_t_seg,min_o_seg,min_t_seg,
        hr_o_seg,hr_t_seg,data;
wire clk1, clk2;
clkdiv cl (clk, clk1);
clkdiv1c2(clk,clk2);
rtcr1(clk1,rst,sec,min,hr,sec_o,sec_t,min_o,min_t,hr_o,
    hr_t,sec_o_seg,sec_t_seg,min_o_seg,min_t_seg,
    hr_o_seg,hr_t_seg);
digit_selectiond1(sec_o_seg,sec_t_seg,min_o_seg,min_t_seg,
            hr_o_seg,hr_t_seg,clk2,rst,digit,data);
endmodule


//RIC
module rtc (clk, rst, sec, min, hr, sec_o, sec_t,
        min_o, min_t, hr_o, hr_t, sec_o_seg,
        sec_t_seg,min_o_seg,min_t_seg,hr_o_seg,
        hr_t_seg);
input clk,rst;
output reg [5:0] sec,min;
output reg [3:0] hr;
outputreg [3:0] sec_o,sec_t,min_o,min_t,hr_o,hr_t;
output reg [7:0] sec_o_seg,sec_t_seg,min_o_seg,min_t_seg;
output reg [7:0] hr_o_seg,hr_t_seg;
always @(posedge clk) begin
   if(rst)
begin
   sec = 6'b0;
   min = 6'b0;
   hr = 4'b0;
end
else
begin
   if(sec == 6'b111101) begin
```

```
            sec = 6'b0;
            min = min + 1;
            if(min == 6'b111101) begin
              min = 6'b0;
              hr = hr + 1;
            end
          end
          else
            sec = sec + 1;
        end
        begin
          if(sec <= 6'b001001)
            begin
              sec_o = sec[3:0];
              sec_t = 4'b0000;
            end
          else if(sec > 6'b001001 && sec <= 6'b010011)
            begin
              sec_t = 4'b0001;
              sec_o = sec - 6'b001010;
            end
          else if(sec > 6'b010011 && sec <= 6'b011101)
            begin
              sec_t = 4'b0010;
              sec_o = sec - 6'b010100;
            end
          else if(sec > 6'b011101 && sec <= 6'b100111)
            begin
              sec_t = 4'b0011;
              sec_o = sec - 6'b011110;
            end
          else if(sec > 6'b100111 && sec <= 6'b110001)
            begin
              sec_t = 4'b0100;
              sec_o = sec - 6'b101000;
            end
          else if(sec > 6'b110001 && sec <= 59)
            begin
              sec_t = 4'b0101;
              sec_o = sec - 6'b110010;
            end
        if(min <= 6'b001001)
          begin
```


dc.v
dc_tb_behav.wcfg
Name
Value
clk
1
rst
0
sec[5 0]
38
min[5 0]
01
hr[4 0]
00
920.000 ns
940.000 ns
960.000 ns
980.000 ns
28
29
30
31
32
33
34
35
36
37
38
01
00

```
        min_o = min[3:0];
        min_t = 4'b0000;
      end
    else if(min > 6'b001001 && min <= 6'b010011)
      begin
        min_t = 4'b0001;
        min_o = min - 6'b001010;
      end
    else if(min > 6'b010011 && min <= 6'b011101)
      begin
        min_t = 4'b0010;
        min_o = min - 6'b010100;
      end
    else if(min > 6'b011101 && min <= 6'b100111)
begin
    min_t=4'b0011;
    min_o=min-6'b011110;
end
else if(min > 6'b100111 && min <= 6'b110001)
begin
    min_t=4'b0100;
    min_o=min-6'b101000;
end
else if(min > 6'b110001 && min <= 59)
begin
    min_t=4'b0101;
    min_o=min-6'b110010;
end
if(hr<=6'b001001)
begin
    hr_o=hr[3:0];
    hr_t=4'b0000;
end
elseif(hr>6'b001001 && hr<=6'b001100)
begin
    hr_t=4'b0001;
    hr_o=hr-4'b1010;
end
end
begin
    case(sec_o)
      4'b0000: sec_o_seg=8'b11000000;
      4'b0001: sec_o_seg=8'b11111001;
```

2403610710622049

```
        4'b0010: sec_o_seg=8'b10100100;
        4'b0011: sec_o_seg=8'b10110000;
        4'b0100: sec_o_seg=8'b10011001;
        4'b0101: sec_o_seg=8'b10010010;
        4'b0110: sec_o_seg=8'b10000010;
        4'b0111: sec_o_seg=8'b11111000;
        4'b1000: sec_o_seg=8'b10000000;
        4'b1001: sec_o_seg=8'b10011000;
      endcase
    case(sec_t)
        4'b0000: sec_t_seg=8'b11000000;
        4'b0001: sec_t_seg=8'b11111001;
        4'b0010: sec_t_seg=8'b10100100;
        4'b0011: sec_t_seg=8'b10110000;
        4'b0100: sec_t_seg=8'b10011001;
        4'b0101: sec_t_seg=8'b10010010;
        4'b0110: sec_t_seg=8'b10000010;
        4'b0111: sec_t_seg=8'b11111000;
        4'b1000: sec_t_seg=8'b10000000;
        4'b1001: sec_t_seg=8'b10011000;
      endcase

    case(min_o)
        4'b0000: min_o_seg=8'b11000000;
        4'b0001: min_o_seg=8'b11111001;
        4'b0010:min_o_seg=8'b10100100;
        4'b0011:min_o_seg=8'b10110000;
        4'b0100:min_o_seg=8'b10011001;
        4'b0101:min_o_seg=8'b10010010;
        4'b0110:min_o_seg=8'b10000010;
        4'b0111:min_o_seg=8'b11111000;
        4'b1000:min_o_seg=8'b10000000;
        4'b1001:min_o_seg=8'b10011000;
     Endcase

    case(min_t)
      4'b0000:min_t_seg=8'b11000000;
      4'b0001:min_t_seg=8'b11111001;
      4'b0010:min_t_seg=8'b10100100;
      4'b0011:min_t_seg=8'b10110000;
      4'b0100:min_t_seg=8'b10011001;
      4'b0101:min_t_seg=8'b10010010;
      4'b0110:min_t_seg=8'b10000010;
```

```
   4'b0111:min_t_seg=8'b11111000;
   4'b1000:min_t_seg=8'b10000000;
   4'b1001:min_t_seg=8'b10011000;
endcase

case(hr_o)
   4'b0000:hr_o_seg=8'b11000000;
   4'b0001:hr_o_seg=8'b11111001;
   4'b0010:hr_o_seg=8'b10100100;
   4'b0011:hr_o_seg=8'b10110000;
   4'b0100:hr_o_seg=8'b10011001;
   4'b0101:hr_o_seg=8'b10010010;
   4'b0110:hr_o_seg=8'b10000010;
   4'b0111:hr_o_seg=8'b11111000;
   4'b1000:hr_o_seg=8'b10000000;
   4'b1001:hr_o_seg=8'b10011000;
 Endcase

 case(hr_t)
   4'b0000:hr_t_seg=8'b11000000;
   4'b0001:hr_t_seg=8'b11111001;
endcase
end
end
endmodule
```

//DIGITSELECTION

```
module digit_selection(sec_o_seg, sec_t_seg,
                       min_o_seg, min_t_seg, hr_o_seg,
                       hr_t_seg, clk, rst, digit, data);
input [7:0] sec_o_seg, sec_t_seg, min_o_seg, min_t_seg, hr_o_seg,
        hr_t_seg;
input clk, rst;
output reg [3:0] digit;
output reg [7:0] data;
reg [1:0] sel;
always @(posedge clk) begin
   if(rst)
      sel = 2'b00;
   else
   begin
```

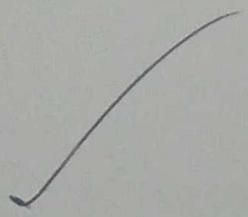
```
        case(sel)
          2'b00: begin digit = 4'b1110; data = sec_o_seg; end
          2'b01: begin digit = 4'b1101; data = sec_t_seg; end
          2'b10: begin digit = 4'b1011; data = min_o_seg; end
          2'b11: begin digit = 4'b0111; data = min_t_seg; end
        endcase
    sel = sel + 2'b01;
    end
end
endmodule
```

//CLKDIV1

```
module clkdiv1(clk,clk2);
input clk;
output reg clk2;
reg[26:0]c;
initial c=27'b0;
always@(posedge clk) begin
    c=c+1;
    clk2=c;
end
endmodule
```

//CLKDIV

```
module clkdiv(clk,clk1);
input clk;
output reg clk1;
reg[10:0]c;
initial c=11'b0;
always@(posedge clk) begin
    c=c+1;
    clk1=c;
end
endmodule
```



RESULT:

To design a digital clock using verilog HDL and to implement it on FPGA was executed and output verified successfully.

2403610710622049

EXP NO: 5 DATE: 28/6/26	VENDING MACHINE USING VERILOG HDL IN VIVADO

AIM:

To design and simulate a Vending Machine using Verilog HDL in Vivado and verify the output using a Finite State Machine (FSM).

SOFTWARE USED:

Vivado 2025.2

ALGORITHM:

Step 1: Open Vivado and create a new RTL project.

Step 2: Create the design source file vending_machine.v.

Step 3: Define the states S0, S5, S10 and S15 representing the amount of money inserted.

Step 4: Write the sequential logic to update the present state on every positive clock edge, with synchronous reset.

Step 5: Write the combinational logic to determine the next state based on the coin input.

Step 6: Write the output logic to assert nw_pa (dispense signal) high only in state S15.

Step 7: Create the simulation source file vending_machine_tb.v.

Step 8: Write the testbench code and apply different coin input values (5 and 10).

Step 9: Run the Behavioral Simulation in Vivado.

Step 10: Observe the waveform and verify that nw_pa goes high once 15 units are reached.

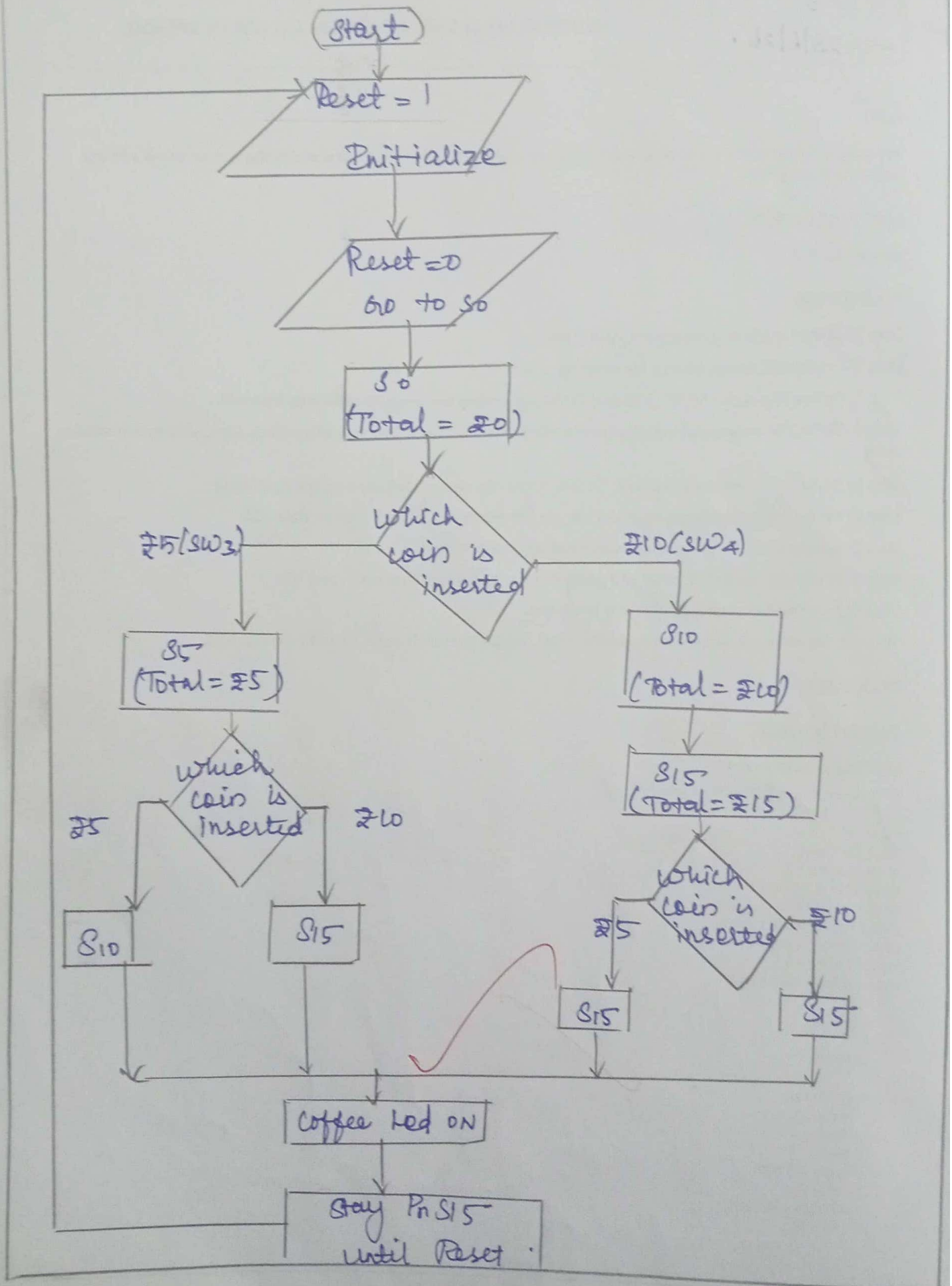
PROGRAM:

DESIGN SOURCE:

vending_machine.v:

```
module vending_machine(clk,rst,coin,nw_pa);
input clk;
input rst;
input [1:0] coin;
output reg nw_pa;
parameter [1:0] S0  = 2'b00;
parameter [1:0] S5  = 2'b01;
parameter [1:0] S10 = 2'b10;
parameter [1:0] S15 = 2'b11;
reg [1:0] State,NS;
always @(posedge clk) begin
  if (rst)
    State=S0;
  else
    State=NS;
end
always @(*) begin
  NS = State;
  case(State)
    S0:begin
      if(coin == 2'b00)
        NS = S0;
      else if (coin == 2'b01)
        NS = S5;
```

FLOWCHART

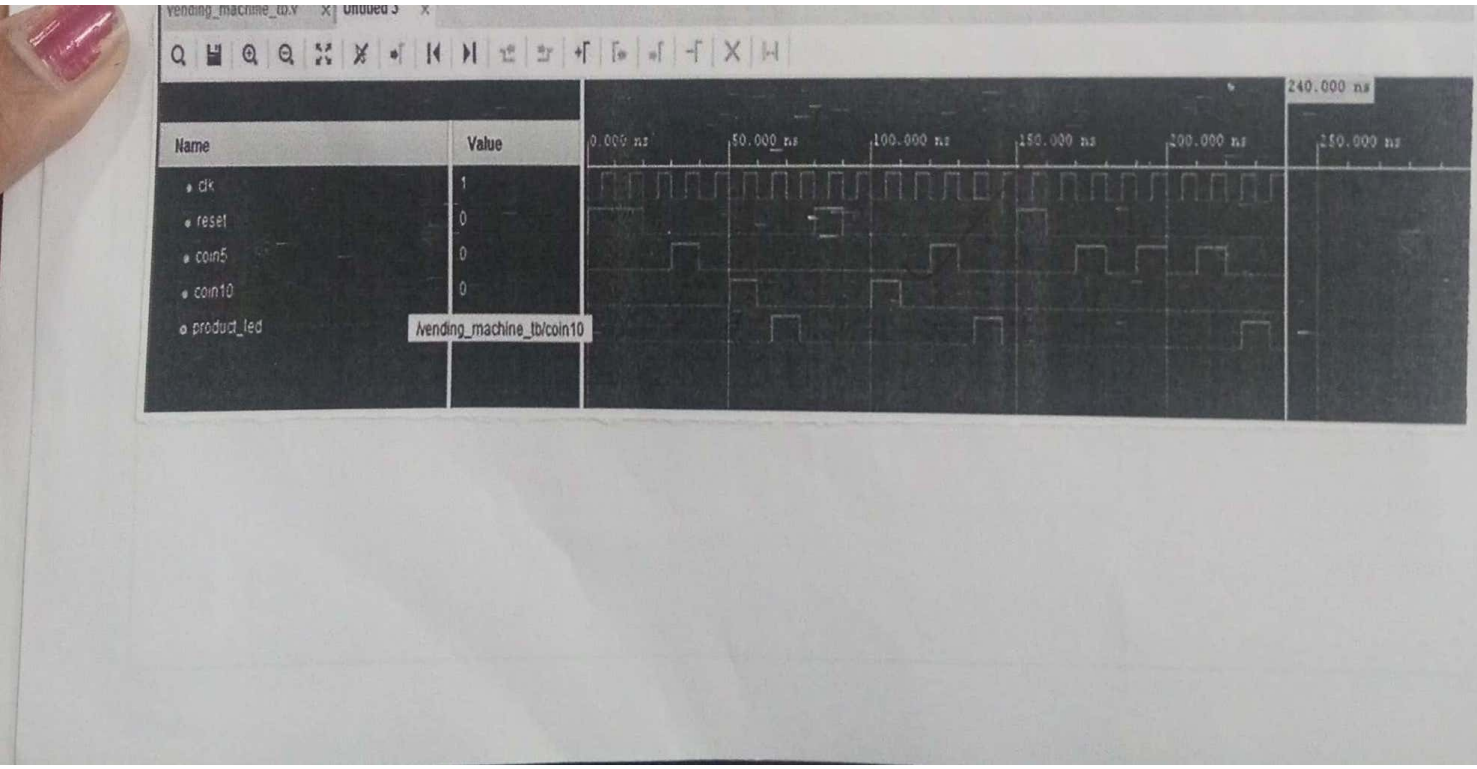

Start
Reset = 1
Initialize
Reset = 0
Go to S0
S0
(Total = ₹0)
Which coin is inserted
₹5 (SW3)
₹10 (SW4)
S5
(Total = ₹5)
S10
(Total = ₹10)
Which coin is inserted
₹5
₹10
S10
S15
S15
(Total = ₹15)
Which coin is inserted
₹5
₹10
S15
S15
Coffee led ON
Stay in S15 until Reset

```
        else
          NS = S10;
      end
      S5:begin
        if(coin == 2'b00)
          NS = S5;
        else if (coin == 2'b01)
          NS = S10;
        else
          NS = S15;
      end
      S10:begin
        if(coin == 2'b00)
          NS = S10;
        else
          NS = S15;
      end
      S15:
        NS = S0;
    endcase
end
always @(State) begin
    case(State)
      S0:  nw_pa = 1'b0;
      S5:  nw_pa = 1'b0;
      S10: nw_pa = 1'b0;
      S15: nw_pa = 1'b1;
    endcase
end
endmodule
```

TESTBENCH CODE:

vending_machine_tb.v:

```
`timescale 1ns/1ps
module vending_machine_tb;
    reg clk;
    reg rst;
    reg [1:0] coin;
    wire nw_pa;
    vending_machine uut (
      .clk(clk),
      .rst(rst),
      .coin(coin),
      .nw_pa(nw_pa)
    );
    always #5 clk = ~clk;
    initial begin
      clk  = 0;
      rst  = 1;
      coin = 2'b00;
      #10;
      rst = 0;
      coin = 2'b00;
      #10;
      coin = 2'b01;
      #10;
      coin = 2'b01;
      #10;
      coin = 2'b01;
      #10;
      coin = 2'b00;
      #10;
      coin = 2'b10;
      #10;
      coin = 2'b01;
```


Vending_machine_tb.v
Untitled 3
Name
Value
clk
reset
coin5
coin10
product_led
1
0
0
0
/vending_machine_tb/coin10
0.000 ns
50.000 ns
100.000 ns
150.000 ns
200.000 ns
250.000 ns
240.000 ns

```
    #10;
    rst = 1;
    coin = 2'b00;
    #10;
    rst = 0;
    #10;
    $finish;
  end
  initial begin
    $monitor("Time=%0t | Reset=%b | Coin=%b | State=%b | nw_pa=%b",
        $time, rst, coin, uut.State, nw_pa);
  end
endmodule
```

RESULT:

Thus the Vending Machine was successfully designed, simulated and verified using Verilog HDL in Vivado.

EXP NO: 6

DATE: 15/9/26

TO READ AN IMAGE AND TO IMPLEMENT IMAGE PROCESSING ALGORITHM

AIM:

To implement 64 × 64 grayscale image processing using Verilog HDL and MATLAB, where an image is converted into grayscale, resized, converted into hexadecimal pixel data, processed using a thresholding operation in Verilog, and reconstructed as a binary image using MATLAB.

SOFTWARE USED:

Vivado 2025.2

MATLAB R2024a

ALGORITHM:

MATLAB – Input Image to HEX

1. Read the peacock image using imread().
2. Convert the RGB image into a grayscale image using rgb2gray().
3. Resize the grayscale image to 64 × 64 pixels using imresize().
4. Convert the image into a one-dimensional pixel array.
5. Open a HEX file named peacock_64x64.hex.
6. Convert each pixel value into an 8-bit hexadecimal value.
7. Store all 4096 pixel values in the HEX file.
8. Close the file.

Verilog – Image Processing

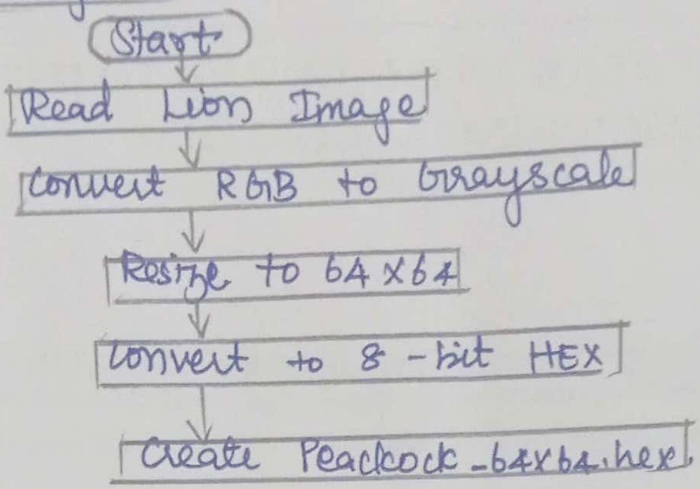
1. Declare an image memory containing 4096 pixels, each of 8 bits.
2. Read peacock_64x64.hex into the image memory using $readmemh().
3. Read each pixel sequentially.
4. Compare each pixel with the threshold value 128.
5. If the pixel value is greater than or equal to 128, assign FF (white).
6. If the pixel value is less than 128, assign 00 (black).
7. Store the processed pixels in the output memory.
8. Write the processed pixel values into output.hex.
9. Display the input and output pixel values.
10. End the simulation.

MATLAB – Output HEX to Image

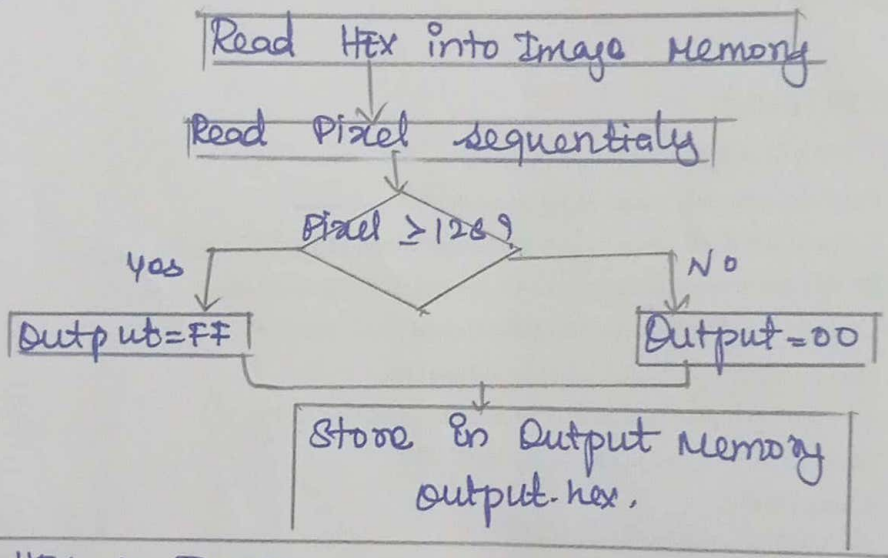
1. Read the generated output.hex file.
2. Remove empty lines.

FLOWCHART:

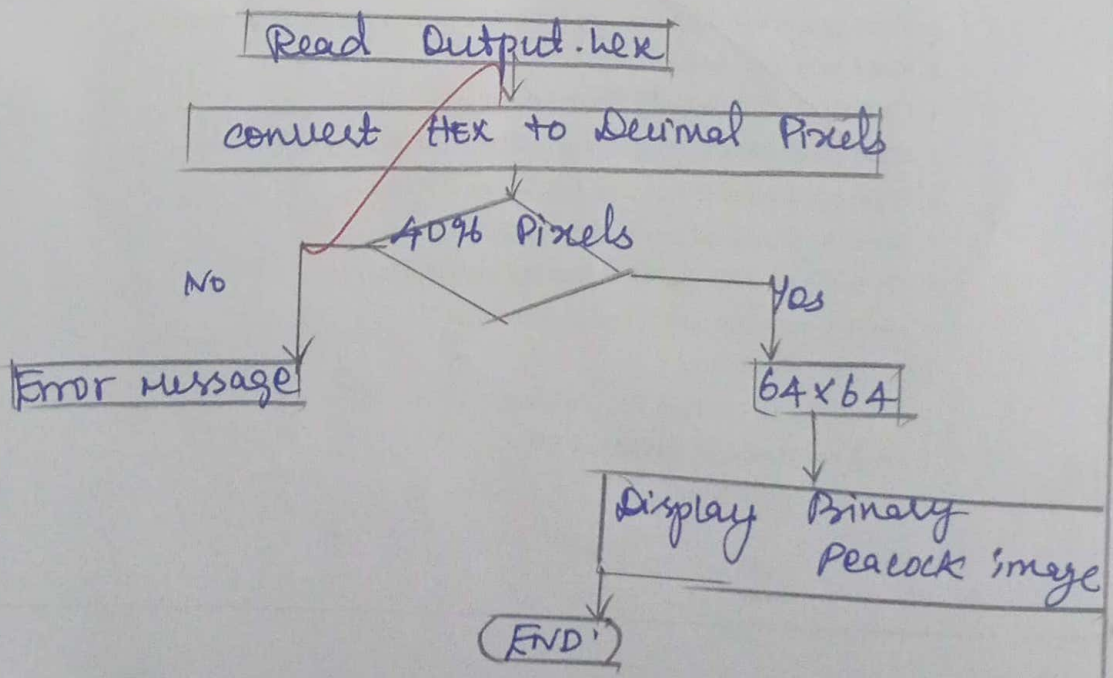
MATLAB: Input Image to HEX



VERILOG: Image Processing



MATLAB-OUTPUT HEX to Image



3. Convert hexadecimal values into decimal pixel values.
4. Verify that exactly 4096 pixels are available.
5. Reshape the pixels into a 64 × 64 matrix.
6. Convert the matrix into an 8-bit image.
7. Display the processed binary image.

PROGRAM:

MATLAB – Generate input HEX file

```
clc;
clear;
close all;
% Read peacock image
img = imread('peacock.jpg');
% Convert RGB image to grayscale
gray_img = rgb2gray(img);
% Resize image to 64 x 64
gray_img = imresize(gray_img, [64 64]);
% Display grayscale image
figure;
imshow(gray_img);
title('64 x 64 Grayscale peacock Image');
% Convert image into pixel array
pixels = gray_img(:);
% Create HEX file
fid = fopen('peacock_64x64.hex', 'w');
for i = 1:length(pixels)
fprintf(fid, '%02X\n', pixels(i));
end
% Read peacock image
img = imread('peacock.jpg');
title('64 x 64 Grayscale peacock Image');
fid = fopen('peacock_64x64.hex', 'w');
fclose(fid);
disp('HEX file generated successfully!');
```

Verilog - Image Threshold Processing

```
module image_processing;
// Input image memory: 4096 pixels, each pixel is 8 bits
reg [7:0] image_mem [0:4095];
// Output image memory
```

```
reg [7:0] output_mem [0:4095];
integer i;
integer outfile;
initial begin
// STEP 1: Read the image HEX file created by MATLAB
$readmemh("peacock_64x64.hex", image_mem);
// STEP 2: Process every pixel
// Pixel >= 128 = WHITE
// Pixel < 128  = BLACK
for(i = 0; i < 4096; i = i + 1) begin
if(image_mem[i] >= 8'd128)
output_mem[i] = 8'hFF;
else
output_mem[i] = 8'h00;
end
// STEP 3: Create output.hex
outfile = $fopen("C:/Users/Public/output.hex", "w");
// STEP 4: Write processed pixels into output.hex
for(i = 0; i < 4096; i = i + 1) begin
$fdisplay(outfile, "%02X", output_mem[i]);
end
// STEP 5: Close the file
$fclose(outfile);
// STEP 6: Display first 4096 pixels in Vivado
for(i = 0; i < 4096; i = i + 1) begin
$display("Pixel[%0d] = %h  Output = %h",
i, image_mem[i], output_mem[i]);
$readmemh("peacock_64x64.hex", image_mem);
end
$display("=====================================");
$display("Processing completed!");
$display("output.hex created successfully!");
$display("=====================================");
$finish;
end
endmodule
```

MATLAB – Display Processed Image

```
clc;
clear;
close all;
% Read output file created by Vivado
data = readlines('output.hex');
```

```
% Remove empty lines
data = strtrim(data);
data(data == "") = [];
% Convert hexadecimal values to decimal
pixels = hex2dec(data);
% Check number of pixels
disp(['Number of pixels = ', num2str(length(pixels))]);
% Make sure we have exactly 4096 pixels
if length(pixels) == 4096
% Convert 4096 pixels into 64 x 64 image
image = reshape(pixels, [64 64]);
% Display image
figure;
imshow(uint8(image));
title('Processed peacock Image');
else
disp('ERROR: output.hex does not contain exactly 4096 pixels.');
end
title('Processed peacock Image');
```

RESULT:

Thus, the given 64 × 64 peacock image was successfully converted into grayscale and hexadecimal pixel data using MATLAB. The pixel data was processed using Verilog HDL with a threshold value of 128, where pixels greater than or equal to 128 were converted to white (FF) and pixels below 128 were converted to black (00). The processed HEX data was successfully reconstructed and displayed as a binary peacock image using MATLAB.

EX NO: 7

DATE: 29/9/26

2403610710622049

TRAFFIC LIGHT SYSTEM

AIM:

To design and simulate a four-way traffic light control system using Verilog HDL, where the traffic signals for East, North, West, and South directions are controlled using a finite state machine and a clock-based timing counter.

SOFTWARE USED:

Vivado 2025.2
Verilog HDL

ALGORITHM:

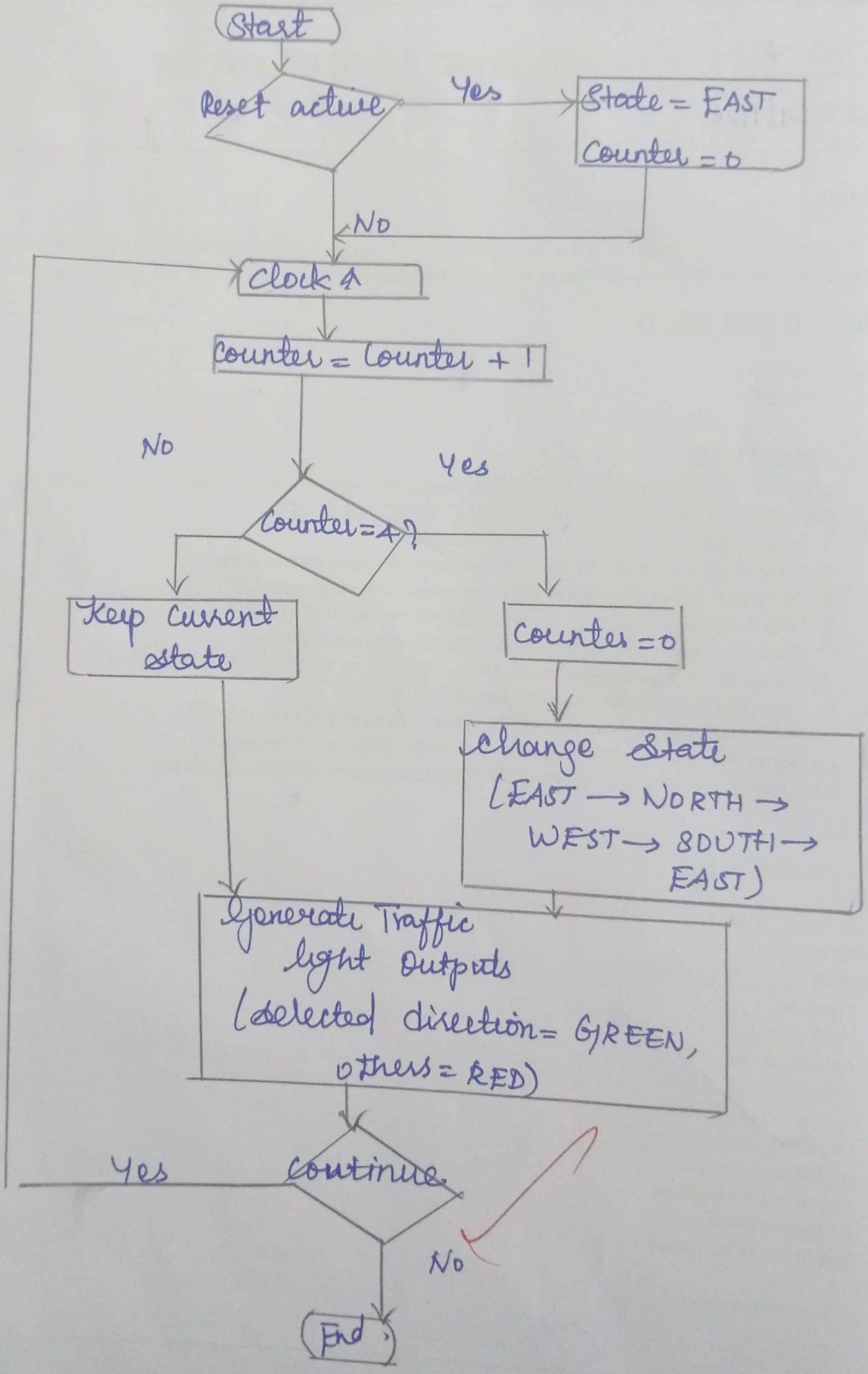
1. Declare clock and reset as the inputs and red/green signals for all four directions as outputs.
2. Define four traffic states: EAST, NORTH, WEST, and SOUTH.
3. Assign a unique 2-bit code to each traffic state.
4. Initialize the state to EAST and the counter to zero when reset is active.
5. On every positive edge of the clock, increment the timing counter.
6. Keep the currently selected direction GREEN for five clock cycles.
7. When the counter reaches the timing limit, reset the counter to zero.
8. Change the state in the sequence EAST → NORTH → WEST → SOUTH → EAST.
9. Use output logic to make all directions RED by default.
10. For the active state, turn its GREEN signal ON and its RED signal OFF.
11. Repeat the state transitions continuously while the system is running.
12. Verify the signal sequence using a Verilog testbench and simulation waveform.

PROGRAM:

SOURCE CODE:

```
`timescale 1ns / 1ps
module traffic_light_4way(
    input clk,
    input reset,
    output reg east_red,
    output reg east_green,
    output reg north_red,
    output reg north_green,
    output reg west_red,
    output reg west_green,
    output reg south_red,
    output reg south_green
);
    // Four traffic states
```

FLOWCHART:


Start
Reset active
Yes
State = EAST
Counter = 0
No
Clock ↑
Counter = Counter + 1
No
Yes
Counter = 4 ?
Keep Current state
Counter = 0
Change State
(EAST → NORTH → WEST → SOUTH → EAST)
Generate Traffic light Outputs
(selected direction = GREEN, others = RED)
Yes
Continue
No
End

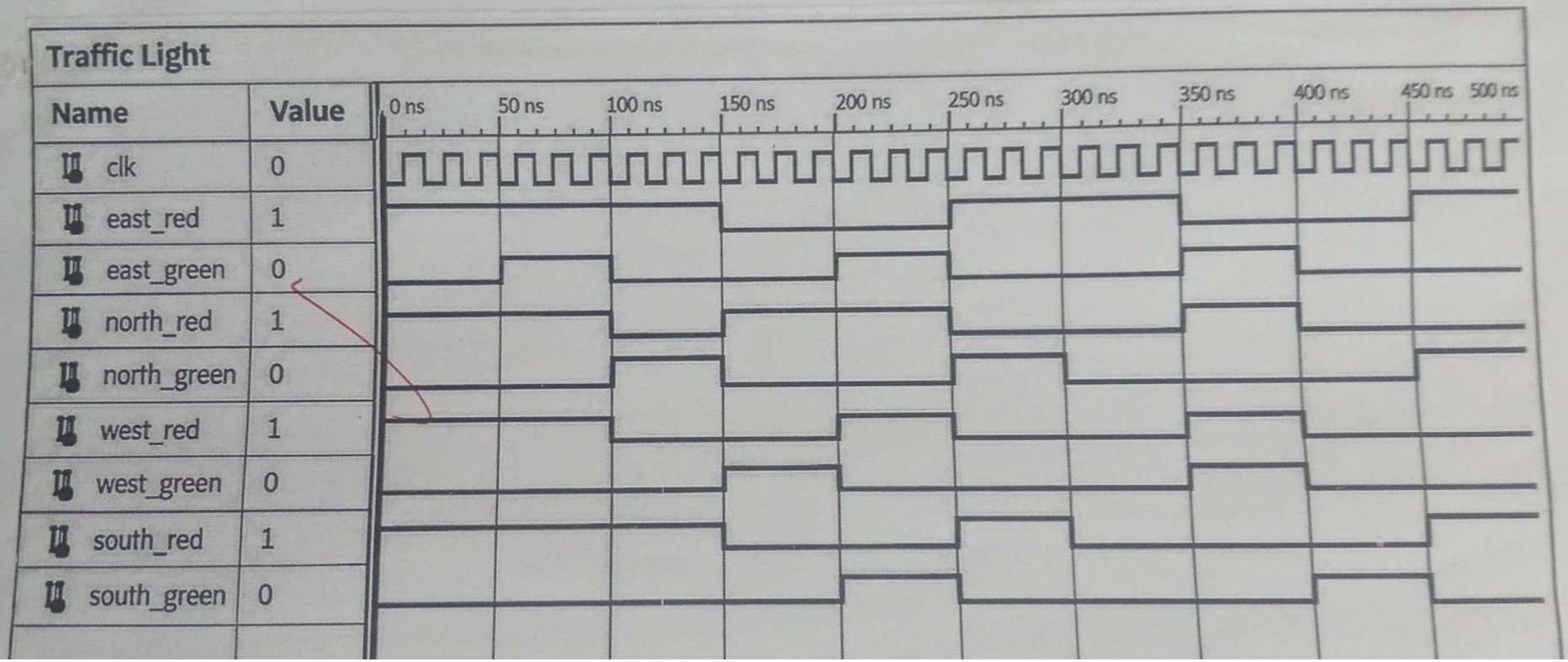
2403610710622049

```
 parameter EAST  = 2'b00 ;
 parameter NORTH = 2'b01;
 parameter WEST  = 2'b10;
 parameter SOUTH = 2'b11;
 reg [1:0] state;
 reg [3:0] count;
// State transition and timing control
always @(posedge clk or posedge reset)
begin
   if (reset)
   begin
      state <= EAST;
      count <= 0;
   end
   else
   begin
 //keep each direction GREEN for 5 clock cycles
      if (count == 4)
      begin
         count <= 0;
         case (state)
            EAST:  state <= NORTH;
            NORTH: state <= WEST;
            WEST:  state <= SOUTH;
            SOUTH: state <= EAST;
            default: state <= EAST;
         endcase
      end
      else
      begin
         count <= count + 1;
      end
   end
end
// Traffic light output logic
always @(*)
begin
   // Default condition: all signals RED
   east_red   = 1;
   east_green  = 0;
   north_red  = 1;
   north_green = 0;
   west_red   = 1
   west_green  = 0;
   south_red  = 1;
   south_green = 0;
   case (state)
```

```
        EAST:
        begin
          east_red   = 0;
          east_green = 1;
        end
        NORTH:
        begin
          north_red   = 0;
          north_green = 1;
        end
        WEST:
        begin
          west_red   = 0;
          west_green = 1;
        end
        SOUTH:
        begin
          south_red   = 0;
          south_green = 1;
        end
      endcase
  end
endmodule
```

TESTBENCH CODE:

```
`timescale 1ns / 1ps
 module traffic_light_4way_tb;
 reg clk;
 reg reset;
   wire east_red, east_green;
   wire north_red, north_green;
   wire west_red, west_green;
   wire south_red, south_green;
   // Instantiate the traffic light controller
   traffic_light_4way dut (
     .clk(clk),
     .reset(reset),
     .east_red(east_red),
     .east_green(east_green),
     .north_red(north_red),
     .north_green(north_green),
     .west_red(west_red),
     .west_green(west_green
     .south_red(south_red),
     .south_green(south_green)
  );
   // Clock generation: 10 ns period
```


Traffic Light
Name
Value
clk
0
east_red
1
east_green
0
north_red
1
north_green
0
west_red
1
west_green
0
south_red
1
south_green
0
0 ns
50 ns
100 ns
150 ns
200 ns
250 ns
300 ns
350 ns
400 ns
450 ns
500 ns

```
Initial
begin
   clk = 0;
   forever #5 clk = ~clk;
end
// Reset and simulation
initial
begin
   reset = 1;
   #20;
   reset = 0;
   #500;
   $finish;
end
// Monitor traffic signal outputs
initial
begin
   $monitor(
   "Time=%0t | E=%b | N=%b | W=%b | S=%b",
   $time,
   east_green,
   north_green,
   west_green,
   south_green
   );
end
endmodule
```

RESULT:

Thus, a four-way traffic light control system was successfully designed and simulated using Verilog HDL. A finite state machine with a clock-based counter was used to control the traffic sequence. Each direction is given GREEN status for five clock cycles while the remaining directions remain RED, and the sequence repeats continuously.

(15)

EX NO: 8

DATE: 29/9/26

2403610710622049

BURGLAR ALARM USING HDL

AIM:

To design a simple burglar alarm system using Verilog HDL in which a siren is activated when any one of the motion sensor, door sensor, or window sensor detects an intrusion, and to simulate and implement the design on an FPGA using Vivado.

APPARATUS REQUIRED:

Hardware: PC, FPGA development kit, motion sensor, door sensor, window sensor, buzzer/siren and connecting wires.

Software: Xilinx Vivado.

ALGORITHM:

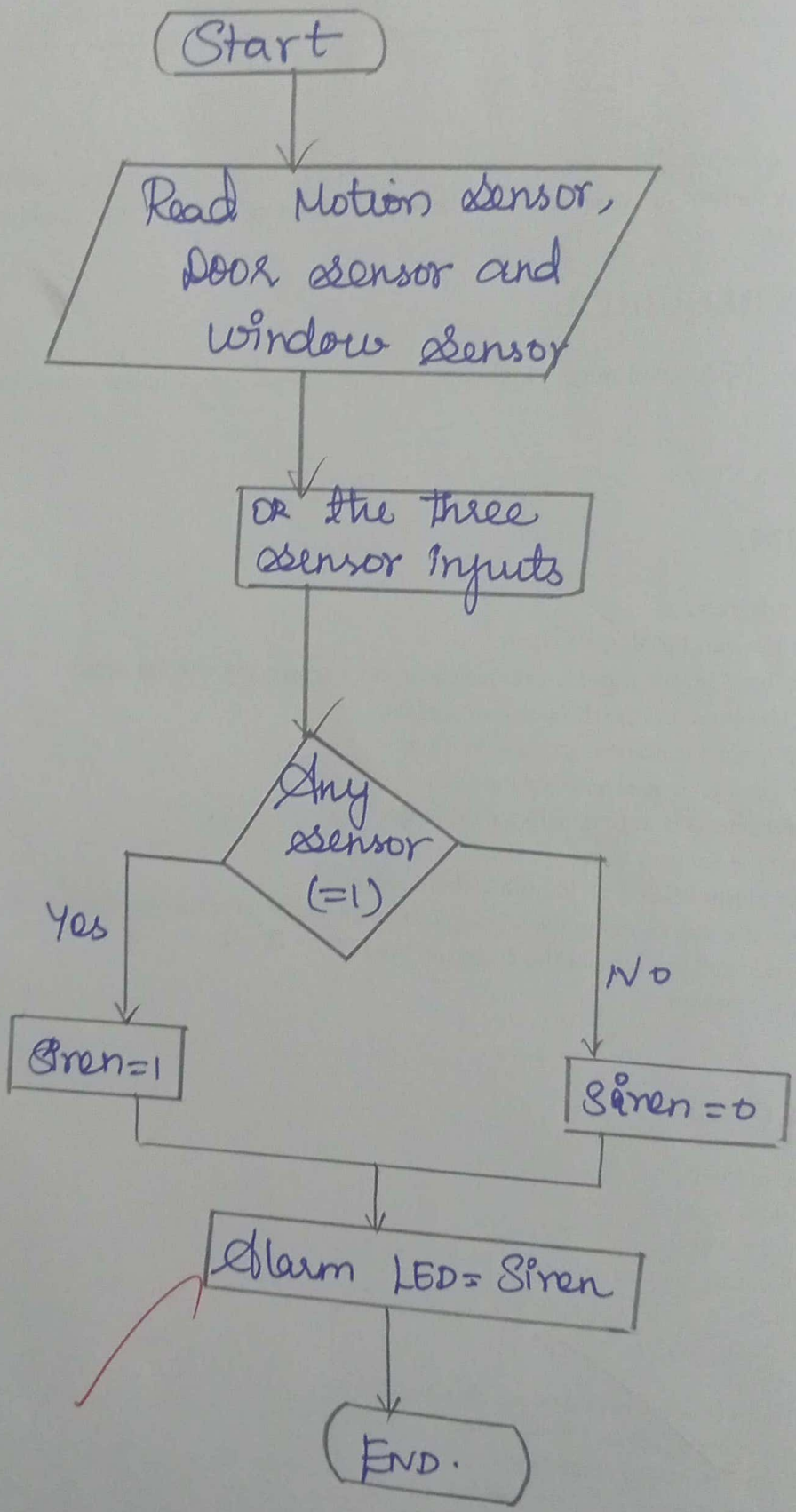
Step 1: Start the program.
Step 2: Initialize the burglar alarm system.
Step 3: Read the three sensor inputs: motion sensor, door sensor and window sensor.
Step 4: Perform OR logic on the three sensor signals.
Step 5: If motion sensor is active, activate the siren.
Step 6: If door sensor is active, activate the siren.
Step 7: If window sensor is active, activate the siren.
Step 8: If none of the sensors is active, keep the siren OFF.
Step 9: Drive the alarm LED with the same alarm signal.
Step 10: Simulate the design using the Verilog testbench and observe the waveform.
Step 11: Synthesize and implement the design on the FPGA.
Step 12: Stop the program.

SOURCE CODE:

```
module burglar_alarm (
    input  wire motion_sensor,
    input  wire door_sensor,
    input  wire window_sensor,
    output wire siren,
    output wire alarm_led
);
    // Siren activates if any sensor detects intrusion
    assign siren = motion_sensor | door_sensor | window_sensor;
    // LED indicates the alarm condition
    assign alarm_led = siren;

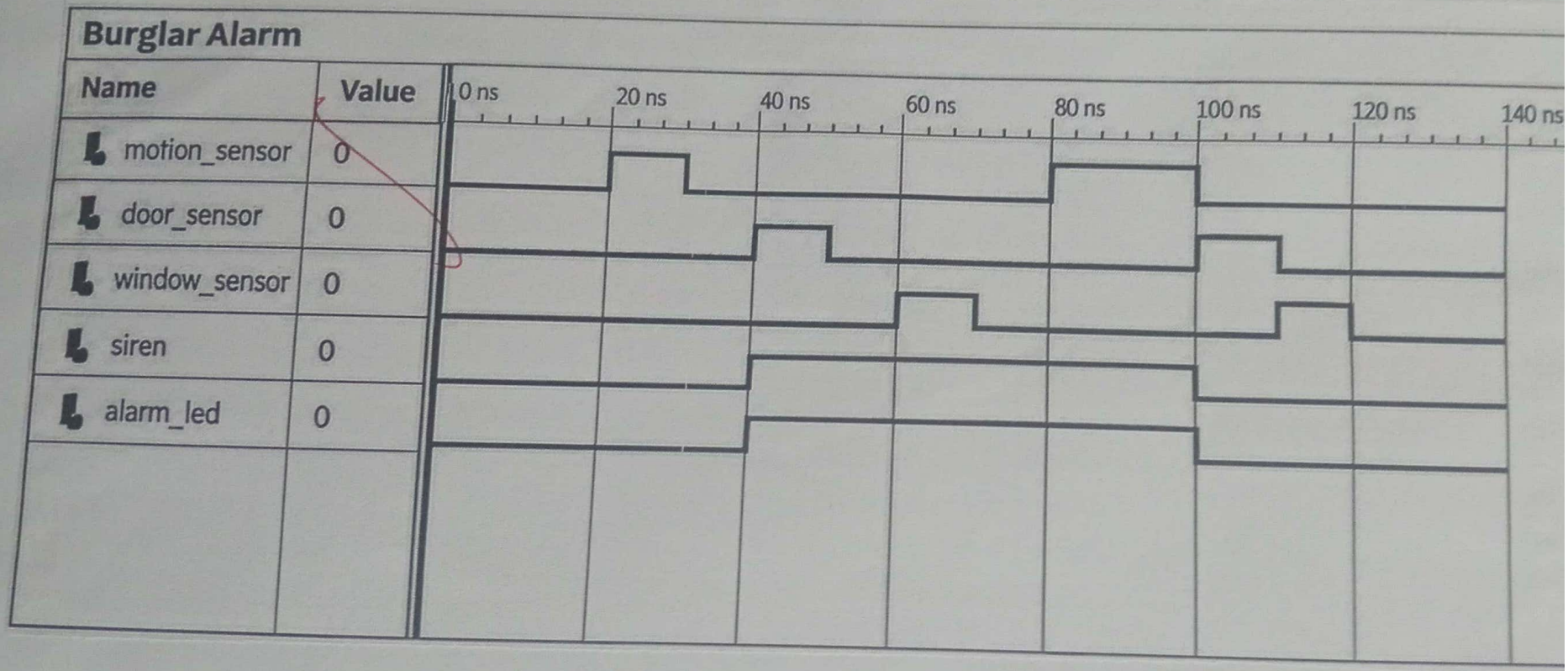
endmodule
```

FLOW CHART :


Start
Read Motion sensor, Door sensor and window sensor
OR the three sensor Inputs
Any sensor (=1)
Yes
No
Siren=1
Siren=0
Alarm LED= Siren
END.

TESTBENCH CODE:

```
timescale 1ns/1ps
module burglar_alarm_tb;
  reg motion_sensor;
  reg door_sensor;
  reg window_sensor;
  wire siren;
  wire alarm_led;
  burglar_alarm uut (
    .motion_sensor(motion_sensor),
    .door_sensor(door_sensor),
    .window_sensor(window_sensor),
    .siren(siren),
    .alarm_led(alarm_led)
);
initial begin
   // Initially all sensors are inactive
   motion_sensor = 0;
   door_sensor   = 0;
   window_sensor = 0;
   #10;
  // Motion detected
   motion_sensor = 1;
   #10;
   // Motion stopped
   motion_sensor = 0;
   #10;
   // Door sensor activated
   door_sensor = 1;
   #10;
   // Door sensor deactivated
   door_sensor = 0;
   #10;
  // Window opened
   window_sensor = 1;
   #10;
   // Window closed
   window_sensor = 0;
   #10;
  // Two sensors active simultaneously
  motion_sensor = 1;
  window_sensor = 1;
  #10;
  // All sensors inactive
  motion_sensor = 0;
  window_sensor = 0;
```


Burglar Alarm
Name
Value
0 ns
20 ns
40 ns
60 ns
80 ns
100 ns
120 ns
140 ns
motion_sensor
0
door_sensor
0
window_sensor
0
siren
0
alarm_led
0

```
    #10;

    $finish;
  end
endmodule
```

IMPLEMENTATION:

```
## Motion Sensor -> SW0
set_property -dict { PACKAGE_PIN L5 IOSTANDARD LVCMOS33 } [get_ports motion_sensor]
## Door Sensor -> SW1
set_property -dict { PACKAGE_PIN L4 IOSTANDARD LVCMOS33 } [get_ports door_sensor]
## Window Sensor -> SW2
set_property -dict { PACKAGE_PIN M4 IOSTANDARD LVCMOS33 } [get_ports window_sensor]
## Siren -> On-board Buzzer
set_property -dict { PACKAGE_PIN K12 IOSTANDARD LVCMOS33 } [get_ports siren]
## Alarm LED -> LED0
set_property -dict { PACKAGE_PIN J3 IOSTANDARD LVCMOS33 } [get_ports alarm_led]
```

RESULT:

Thus, the burglar alarm was designed using Verilog HDL. The simulation verified that the siren and alarm LED are activated whenever the motion sensor, door sensor, or window sensor is active. The design is suitable for synthesis and FPGA implementation using Vivado.

EX NO: 9

DATE: 6/10/26

TIC-TAC-TOE GAME USING VERILOG HDL

AIM:

To design a digital Tic-Tac-Toe game using Verilog HDL in which two players take alternate turns by selecting one of the nine cells, detect the winning combination or draw condition, and implement the design on an FPGA using Vivado.

APPARATUS REQUIRED:

Hardware: PC, FPGA development kit with clock, push buttons/switches and LEDs, and connecting wires.

Software: Xilinx Vivado.

ALGORITHM:

1. Start the program.
2. Initialize the Tic-Tac-Toe board and set Player X as the first player.
3. Read the reset input and the nine cell-selection push buttons.
4. If reset is active, clear all nine cells and restart the game with Player X.
5. Detect a valid button press for an empty cell.
6. Store the current player's symbol in the selected cell.
7. Check all possible winning combinations: three rows, three columns and two diagonals.
8. If a winning combination is detected, indicate the winner and stop accepting further moves.
9. If all nine cells are occupied without a winner, indicate a draw and stop accepting further moves.
10. If the game is not over, change the turn from Player X to Player O or from Player O to Player X.
11. Drive the X and O LED outputs according to the contents of the nine cells.
12. Simulate the design using the Verilog testbench and observe the waveform.
13. Synthesize and implement the design on the FPGA.
14. Stop the program.

PROGRAM:

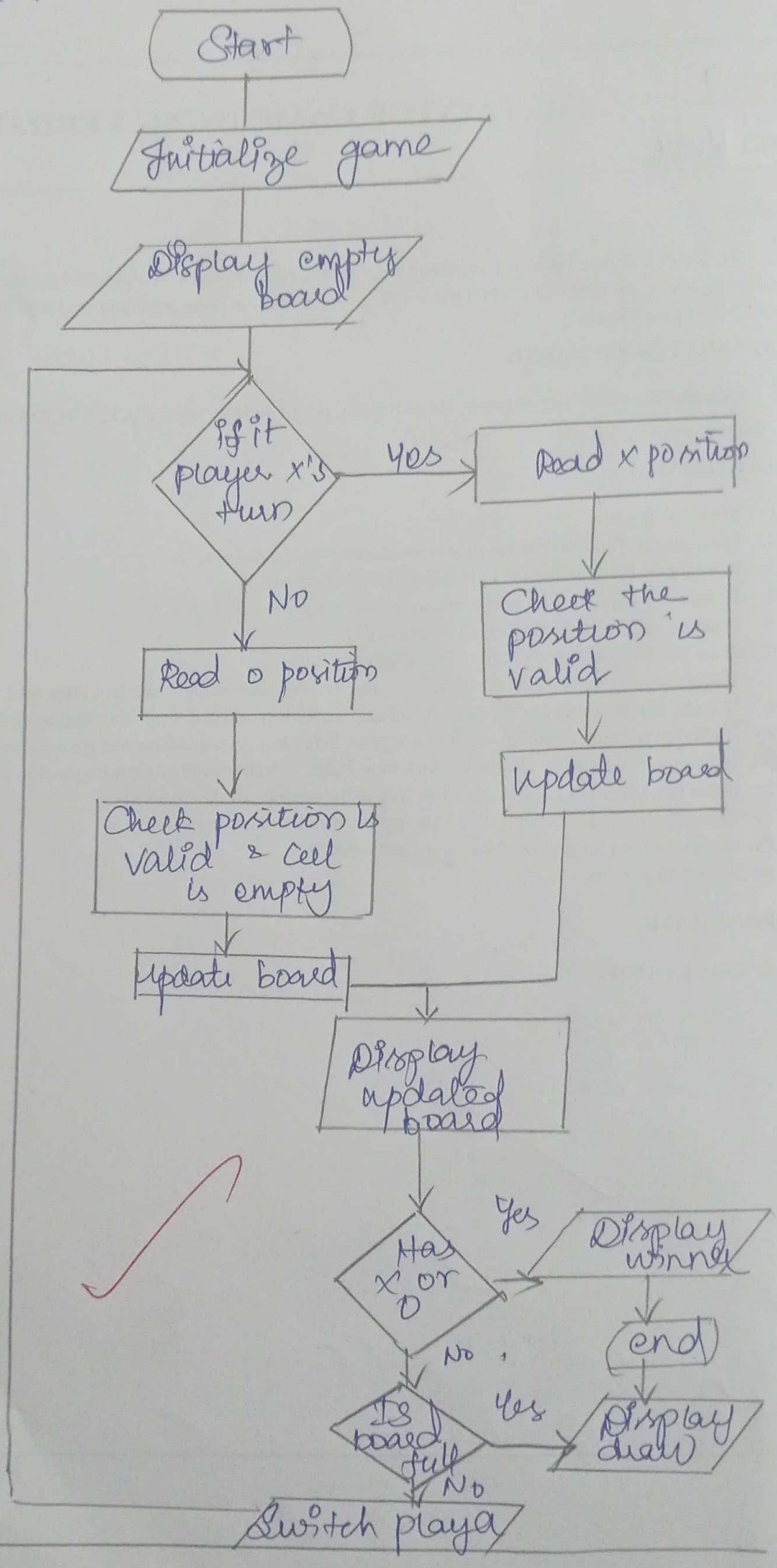
SOURCE CODE:

```
module tic_tac_toe (
    input  wire       clk,
    input  wire       reset,
    input  wire [8:0] cell_btn,

    output reg  [8:0] x_led,
    output reg  [8:0] o_led,
    output reg        win_x,
    output reg        win_o,
    output reg        draw
);

    // board[i] = 0: empty, 1: X, 2: O
    reg [1:0] board [0:8];
    reg       current_player;
    reg       game_over;
```

FLOW CHART:


Start
Initialize game
Display empty board
If it player x's turn
Yes
Read x position
No
Read o position
Check the position is valid
Update board
Check position is valid & cell is empty
Update board
Display updated board
Has X or O
Yes
Display winner
end
No
Is board full
Yes
Display draw
No
Switch player

```
integer i;

// Previous button state for rising-edge detection
reg [8:0] btn_prev;
wire [8:0] btn_rise = cell_btn & ~btn_prev;

function check_win;
    input [1:0] p;
    begin
        check_win =
            ((board[0] == p) && (board[1] == p) && (board[2] == p)) ||
            ((board[3] == p) && (board[4] == p) && (board[5] == p)) ||
            ((board[6] == p) && (board[7] == p) && (board[8] == p)) ||
            ((board[0] == p) && (board[3] == p) && (board[6] == p)) ||
            ((board[1] == p) && (board[4] == p) && (board[7] == p)) ||
            ((board[2] == p) && (board[5] == p) && (board[8] == p)) ||
            ((board[0] == p) && (board[4] == p) && (board[8] == p)) ||
            ((board[2] == p) && (board[4] == p) && (board[6] == p));
    end
endfunction

always @(posedge clk) begin
    btn_prev <= cell_btn;

    if (reset) begin
        for (i = 0; i < 9; i = i + 1)
            board[i] <= 2'b00;

        x_led          <= 9'b0;
        o_led          <= 9'b0;
        win_x          <= 1'b0;
        win_o          <= 1'b0;
        draw           <= 1'b0;
        current_player <= 1'b1;   // X starts
        game_over      <= 1'b0;
    end
    else if (!game_over) begin
        // Check each cell for a new button press
        for (i = 0; i < 9; i = i + 1) begin
            if (btn_rise[i] && (board[i] == 2'b00)) begin

                if (current_player == 1'b1)
                    board[i] <= 2'b01;   // X
                else
                    board[i] <= 2'b10;   // O

                // Update display
                if (current_player == 1'b1)
                    x_led[i] <= 1'b1;
                else
                    o_led[i] <= 1'b1;

                // Check for winner using the newly selected cell
                // The board state is updated on the next clock edge,
                // so winner indication is checked in the following cycle.

                if (current_player == 1'b1)
                    current_player <= 1'b0;
                else
```

```
                    current_player <= 1'b1;
                end
            end

            // Winner detection
            if (check_win(2'b01)) begin
                win_x     <= 1'b1;
                game_over <= 1'b1;
            end
            else if (check_win(2'b10)) begin
                win_o     <= 1'b1;
                game_over <= 1'b1;
            end
            else if ((board[0] != 0) && (board[1] != 0) &&
                     (board[2] != 0) && (board[3] != 0) &&
                     (board[4] != 0) && (board[5] != 0) &&
                     (board[6] != 0) && (board[7] != 0) &&
                     (board[8] != 0)) begin
                draw      <= 1'b1;
                game_over <= 1'b1;
            end
        end
    end

endmodule
```

TESTBENCH CODE:

```
`timescale 1ns/1ps

module tic_tac_toe_tb;

    reg         clk;
    reg         reset;
    reg [8:0]   cell_btn;

    wire [8:0] x_led;
    wire [8:0] o_led;
    wire       win_x;
    wire       win_o;
    wire       draw;

    tic_tac_toe uut (
        .clk(clk),
        .reset(reset),
        .cell_btn(cell_btn),
        .x_led(x_led),
        .o_led(o_led),
        .win_x(win_x),
        .win_o(win_o),
        .draw(draw)
    );

    // 10 ns clock
    always #5 clk = ~clk;

    // Press one cell for one clock period
    task press_cell;
        input integer n;
```

```
            begin
                cell_btn[n] = 1'b1;
                #10;
                cell_btn[n] = 1'b0;
                #10;
            end
        endtask

        initial begin
            clk      = 1'b0;
            reset    = 1'b1;
            cell_btn = 9'b0;

            #20;
            reset = 1'b0;

            // X: cells 0, 1, 2 -> X wins
            press_cell(0);
            press_cell(3);
            press_cell(1);
            press_cell(4);
            press_cell(2);

            #20;

            // Reset and test another game if required
            reset = 1'b1;
            #10;
            reset = 1'b0;

            // X: 0,4,8 -> diagonal win
            press_cell(0);
            press_cell(1);
            press_cell(4);
            press_cell(2);
            press_cell(8);

            #20;
            $finish;
        end

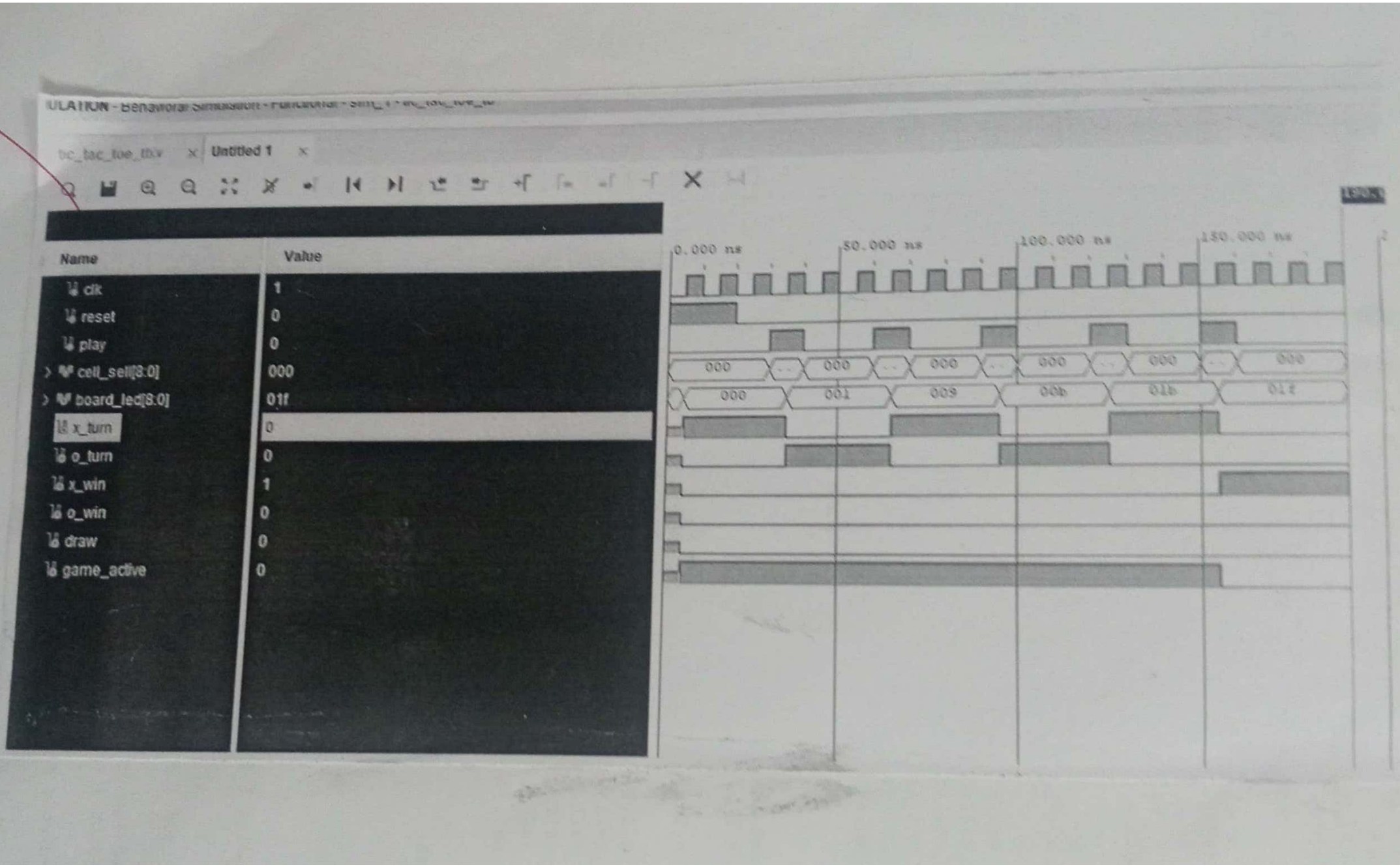
    endmodule
```

Tic-tac-toe.xdc:

```
## Clock
set_property -dict { PACKAGE_PIN <CLK_PIN> IOSTANDARD LVCMOS33 } [get_ports clk]

## Reset
set_property -dict { PACKAGE_PIN <RESET_PIN> IOSTANDARD LVCMOS33 } [get_ports reset]

## Nine Tic-Tac-Toe cell buttons
set_property -dict { PACKAGE_PIN <CELL0_PIN> IOSTANDARD LVCMOS33 } [get_ports
{cell_btn[0]}]
set_property -dict { PACKAGE_PIN <CELL1_PIN> IOSTANDARD LVCMOS33 } [get_ports
{cell_btn[1]}]
set_property -dict { PACKAGE_PIN <CELL2_PIN> IOSTANDARD LVCMOS33 } [get_ports
{cell_btn[2]}]
set_property -dict { PACKAGE_PIN <CELL3_PIN> IOSTANDARD LVCMOS33 } [get_ports
```


Untitled 1
Name
Value
clk
1
reset
0
play
0
cell_sel[8:0]
000
board_lec[8:0]
01f
x_turn
0
o_turn
0
x_win
1
o_win
0
draw
0
game_active
0
0.000 ns
50.000 ns
100.000 ns
150.000 ns
000
001
009
00b
01b
01f

```
{cell_btn[3]}]
set_property -dict { PACKAGE_PIN <CELL4_PIN> IOSTANDARD LVCMOS33 } [get_ports
{cell_btn[4]}]
set_property -dict { PACKAGE_PIN <CELL5_PIN> IOSTANDARD LVCMOS33 } [get_ports
{cell_btn[5]}]
set_property -dict { PACKAGE_PIN <CELL6_PIN> IOSTANDARD LVCMOS33 } [get_ports
{cell_btn[6]}]
set_property -dict { PACKAGE_PIN <CELL7_PIN> IOSTANDARD LVCMOS33 } [get_ports
{cell_btn[7]}]
set_property -dict { PACKAGE_PIN <CELL8_PIN> IOSTANDARD LVCMOS33 } [get_ports
{cell_btn[8]}]

## X LEDs
set_property -dict { PACKAGE_PIN <X0_PIN> IOSTANDARD LVCMOS33 } [get_ports {x_led[0]}]
set_property -dict { PACKAGE_PIN <X1_PIN> IOSTANDARD LVCMOS33 } [get_ports {x_led[1]}]
set_property -dict { PACKAGE_PIN <X2_PIN> IOSTANDARD LVCMOS33 } [get_ports {x_led[2]}]
set_property -dict { PACKAGE_PIN <X3_PIN> IOSTANDARD LVCMOS33 } [get_ports {x_led[3]}]
set_property -dict { PACKAGE_PIN <X4_PIN> IOSTANDARD LVCMOS33 } [get_ports {x_led[4]}]
set_property -dict { PACKAGE_PIN <X5_PIN> IOSTANDARD LVCMOS33 } [get_ports {x_led[5]}]
set_property -dict { PACKAGE_PIN <X6_PIN> IOSTANDARD LVCMOS33 } [get_ports {x_led[6]}]
set_property -dict { PACKAGE_PIN <X7_PIN> IOSTANDARD LVCMOS33 } [get_ports {x_led[7]}]
set_property -dict { PACKAGE_PIN <X8_PIN> IOSTANDARD LVCMOS33 } [get_ports {x_led[8]}]

## O LEDs
set_property -dict { PACKAGE_PIN <O0_PIN> IOSTANDARD LVCMOS33 } [get_ports {o_led[0]}]
set_property -dict { PACKAGE_PIN <O1_PIN> IOSTANDARD LVCMOS33 } [get_ports {o_led[1]}]
set_property -dict { PACKAGE_PIN <O2_PIN> IOSTANDARD LVCMOS33 } [get_ports {o_led[2]}]
set_property -dict { PACKAGE_PIN <O3_PIN> IOSTANDARD LVCMOS33 } [get_ports {o_led[3]}]
set_property -dict { PACKAGE_PIN <O4_PIN> IOSTANDARD LVCMOS33 } [get_ports {o_led[4]}]
set_property -dict { PACKAGE_PIN <O5_PIN> IOSTANDARD LVCMOS33 } [get_ports {o_led[5]}]
set_property -dict { PACKAGE_PIN <O6_PIN> IOSTANDARD LVCMOS33 } [get_ports {o_led[6]}]
set_property -dict { PACKAGE_PIN <O7_PIN> IOSTANDARD LVCMOS33 } [get_ports {o_led[7]}]
set_property -dict { PACKAGE_PIN <O8_PIN> IOSTANDARD LVCMOS33 } [get_ports {o_led[8]}]

## Game status LEDs
set_property -dict { PACKAGE_PIN <WINX_PIN> IOSTANDARD LVCMOS33 } [get_ports win_x]
set_property -dict { PACKAGE_PIN <WINO_PIN> IOSTANDARD LVCMOS33 } [get_ports win_o]
set_property -dict { PACKAGE_PIN <DRAW_PIN> IOSTANDARD LVCMOS33 } [get_ports draw]
```

RESULT:

Thus, the Tic-Tac-Toe game was designed using Verilog HDL. The design provides alternate turns for two players, stores the selected cells, detects all possible winning combinations and identifies a draw condition. The design can be simulated in Vivado and synthesized for FPGA implementation.